

IC Power Modeling in the Era of Machine Learning: Where Do We Stand?

TAVIA PLATTENTEICH*, Institute of Computer Engineering, Universität zu Lübeck, Germany

PHILIPP FENGLER*, Chair of Electronic Design Automation, Technical University of Munich, Germany

MLADEN BEREKOVIC, Institute of Computer Engineering, Universität zu Lübeck, Germany

ULF SCHLICHTMANN, Chair of Electronic Design Automation, Technical University of Munich, Germany

SALEH MULHEM, Institute of Computer Engineering, Universität zu Lübeck, Germany

Precise power estimation is crucial in IC design, as errors cause thermal violations or energy inefficiency. This paper presents a systematic literature review (SLR) of IC power modeling research published in the past decade. We track three co-evolving dimensions, synthesizing numerous studies: (1) power modeling at three abstraction levels: high-level, register transfer level (RTL), and gate-level netlist; (2) a range of methods from analytical to data-driven ML approaches; and (3) the development of open-source frameworks supporting power modeling. Our SLR identifies unresolved challenges: limited availability of training datasets, insufficient cross-technology and cross-circuit generalization, and the lack of accessible open-source power models.

1 Introduction

Power dissipation is a central design constraint for modern integrated circuits (ICs). In battery-powered edge devices, the energy drawn from a finite battery determines the operational lifetime and dictates the required energy budget for the entire system. In large data-center servers, millions of physical cores generate significant aggregate heat, challenging the thermal management system of a rack and raising both thermal-design and reliability concerns [3]. Consequently, accurate estimation and control of power dissipation are indispensable for design optimization, power supply budgeting, and the mitigation of thermal hotspots. Here, power modeling builds the foundation for such estimation. It maps a set of design-time input parameters to estimates of dynamic and static power. These parameters include clock frequency and activity factors, supply voltage, process-corner details, and architectural choices. Importantly, the mapping is often nonlinear and highly dependent on the abstraction level at which the modeling is performed. Transistor level offers more detailed insights into IC power characteristics compared to higher levels. However, circuit simulation at transistor level is computationally expensive in terms of both runtime and memory consumption. Thus, the state-of-the-art (SoA) power-modeling approaches primarily target high-level (including the electronic system level (ESL) and algorithmic level), Register-Transfer Level (RTL), and gate-level during the earliest stages of IC design. Transitioning between these three levels gives designers varying levels of detail about the underlying design and the targeted technology nodes, shaping the effectiveness and granularity of power modeling. IC designers working at high-level have architectural descriptions and only a coarse picture of the underlying technology nodes, but decisions at this level have significant effect on the design. At the RTL, the designer has a clearer view of signal activity, state-machine behavior, and data-path structure, enabling power modeling with a valuable intermediate level of precision that balances estimation time with fidelity. While high-level and RTL are technology-agnostic, gate-level power models have access to technology-specific

*Both authors contributed equally to this research.

parameters such as transistor sizes, threshold voltages, gate timing, and leakage characteristics. However, gate-level simulation is a computational challenge and exhibits significant runtime overhead compared to RTL and high-level.

Over the past decade, machine-learning (ML) techniques have received significant attention across the entire IC design stack [67]. In particular, power modeling can benefit from these data-driven methods. ML methods have delivered unprecedented accuracy for larger designs, reduced model calibration effort and simulation time, and enabled rapid exploration of vast design spaces that were previously intractable with traditional approaches such as analytical models or LUTs. Yet, the very same properties that make ML attractive also pose distinct challenges: data quality and availability, model interpretability, generalization across technologies, and the integration of ML models into existing design flows. Numerous studies have explored ML for power modeling, but many focus on niche aspects, use incomparable datasets, or replicate previously reported studies without offering new insights.

Contributions. We conduct a systematic literature review (SLR) that addresses five crucial aspects of power modeling. (1) We focus on the critical gap between the classical and ML-based power modeling methods. (2) We deliver the first comprehensive, methodologically rigorous survey of ML-driven power-modeling across the three main abstraction levels (high-level, RTL, and gate-level) of IC design. (3) Here, we focus especially on the fidelity of ground-truth data and its implications for modeling accuracy. (4) To go beyond the SoA surveys, we also cover the role of *open-source* frameworks and toolchains in power modeling. While such frameworks promise transparency, reproducibility, and community collaboration, they face significant design dependencies, limited transferability, and limited support for ML-based power models. (5) We identify these and further open challenges in power modeling.

Table 1. Comparison of Power Modeling Surveys

Study	Abstraction Level			Type of Review		Time frame
	High	Register-Transfer	Gate	SLR	Survey	
[57]	-	✓	-	-	✓	1992-2020
[96]	-	✓	✓	-	✓	2000-2022
[52]	✓	-	-	-	✓	2014-2024
Our	✓	✓	✓	✓	-	2015-2025

To position our work clearly and show its novelty, Table 1 compares surveys on power estimation with our work. While Nasser et al. [57] focus on RTL power modeling for FPGAs and ASICs, Zoni et al. [96] compare different run-time power monitor models. Mahmoudi et al. [52] consider power estimation besides other capabilities at a high abstraction level, such as fault propagation modeling.

Our work elucidates tangible performance gains and intrinsic drawbacks of ML approaches and how level and feature extraction play a bigger role than specific ML models. We also demonstrate how community-driven open-source initiatives have the opportunity to accelerate the diffusion of power-modeling methods. To the best of our knowledge, there exist no surveys that consider gate-level, RTL, and high-level power estimation approaches, and show the evidence of research trends, focusing on system integration and ML usage.

Organization. This manuscript is organized into seven sections as follows: Section 2 presents the fundamentals of circuit power and modeling, including the key error metrics and ML techniques. Section 3 details the SLR methodology, listing our research questions and the two-phase search and screening process to build the evidence base. Section 4 introduces a taxonomy of power modeling techniques and summarizes the major research trends in the field. Section 5 delves into the power-modeling methods themselves, organized by abstraction level and power modeling input

parameters. Section 6 discusses the findings, including aspects, like the growing importance of data provenance and open research challenges. Section 7 summarizes the main insights of this SLR.

2 Background on Power and Modeling

This section highlights the fundamentals of the IC power, the IC development process, power modeling approaches, and the relevant mathematical metrics for modeling.

2.1 IC Power Modeling Concept

2.1.1 Fundamentals in IC Power Dissipation. The total power dissipation P_{total} of a digital integrated complementary metal-oxide-semiconductor (CMOS) circuit is a combination of dynamic power P_{dyn} and static power P_{stat} dissipation. Hence, P_{total} can be calculated as follows [66]:

$$\begin{aligned} P_{total} &= P_{dyn} + P_{stat} \\ &= \alpha(C_L + C_{sc})V_{DD}^2f + V_{DD}I_{stat} \end{aligned} \quad (1)$$

where, C_L is the capacitive load, V_{DD} is the supply voltage, C_{sc} is the short-circuit load capacitance, I_{stat} is the leakage current, and f is the clock frequency. Here, α is the rate of switching cycles of transistors, which contributes to the dynamic power dissipation P_{dyn} . It is noteworthy that P_{total} is determined by a combination of technology-dependent ($C_L, C_{sc}, I_{stat}, V_{DD}$) and technology-independent (f, α) parameters. The availability of technology-dependent parameters varies from one abstraction level to another, posing significant challenges for accurate IC power modeling. Figure 1 shows the structural domain of the abstraction levels in IC design based on the well-known chart of Daniel Gajski and Robert Kuhn [23]. The structural domain defines the assembly of (sub-)circuits to form the digital system at commonly used abstraction levels. The functionality of the (sub-)circuits is described as a behavioral domain at RTL and realized by CMOS transistors at the physical level. Every abstraction level has a different structure size and is often used in different stages of the IC design.

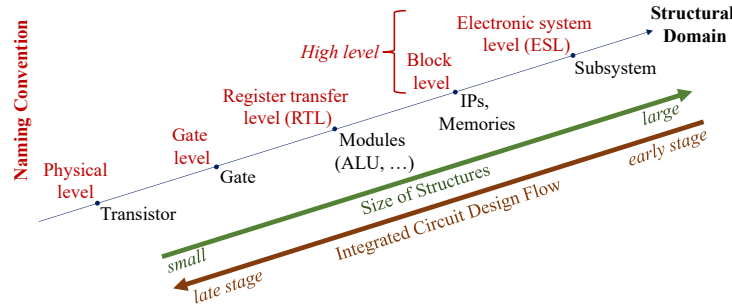


Fig. 1. Structural Domain of Gajski-Kuhn chart on hierarchical circuit design [23]

2.1.2 IC Design Flow. A typical electronic design automation (EDA) flow begins with the specification of the circuit's functionality. This specification can be implemented at Register Transfer Level (RTL) using a hardware description language (HDL) such as Verilog or VHDL. RTL describes the circuit functionality in modules and consists of register stages with combinational circuits in between. The RTL design is then synthesized into a gate-level representation using a standard cell library while optimizing the type and number of gates. A standard cell library provides different types of logic gates and their physical characterization. The synthesized RTL design results in a gate-level netlist. A

netlist is a textual description of the circuit design, the gates, and their connections. Afterward, all netlist components are placed and mapped onto the chip and routed to optimize performance, area, and power consumption. This place and route process results in the chip's physical layout. At this final stage, once the design is validated, it proceeds to tape-out and fabrication. It is worth noting that the recent trend in EDA focuses on high-level synthesis (HLS) tools, which use a standard programming language such as C++/SystemC to describe the circuit at high-level (especially at ESL). High-level code can be analyzed and translated into RTL code for further processing up to IC tape-out and fabrication. As EDA tools can simulate circuit designs at each step from RTL to the physical layout level, they can provide preliminary power estimation reports. This is called design-time power estimation. Power estimation relies on how to evaluate the value of P_{total} . For instance, frequency, voltage, and capacitance are technology-dependent and are usually only provided at gate-level or lower levels. This makes the P_{total} evaluation at higher levels less accurate. Therefore, higher abstraction levels often focus on switching activities and hence model only P_{dyn} .

To accurately estimate power consumption, vector-based IC power modeling has been proposed, where the circuit is simulated with inputs at a specific abstraction level. Here, a Value Change Dump (VCD) file logs information about the switching activities over time of inputs/outputs, registers, gates, or transistors, and is often used as an input feature to power models. On one hand, the size of the VCD file varies greatly, e.g., logging every change of each gate at gate-level versus fewer corresponding register value changes at RTL. On the other hand, gate-level simulations for power design verification are still costly with respect to time and required computational resources, due to the large number of gates in modern circuit designs. The higher the abstraction level, the less information about the hardware is available. This makes the abstraction level a crucial factor in IC power modeling. Therefore, power models at higher abstraction levels are usually faster due to a lower data volume. This results in a trade-off between accuracy and estimation speed.

Another technique deploys vectorless IC power modeling to provide an averaged power model. It assigns default signal rates and static probabilities for input switching behavior.

2.1.3 Error Metrics in Power Modeling. Accuracy is the major property of a power modeling approach. It is evaluated by the error between the model's estimate, P_{est} , and the ground truth data, P_{gt} . Table 2 shows an overview of typical error metrics and their respective formulas for calculation. For a single value, like the average power dissipation of a

Table 2. Formulas for Error Metrics

Single Error Metric	Formula	Multiple Error Metric	Formula
Error (E)	$P_{est} - P_{gt}$	Mean absolute error (MAE)	$\frac{1}{N} \sum_{n=1}^N P_{est,n} - P_{gt,n} $
Percentage error (PE)	$\frac{P_{est} - P_{gt}}{P_{gt}} \cdot 100\%$	Mean absolute percentage error (MAPE)	$\frac{1}{N} \sum_{n=1}^N \frac{ P_{est,n} - P_{gt,n} }{P_{gt,n}} \cdot 100\%$
Absolute error (AE)	$ P_{est} - P_{gt} $	Mean squared error (MSE)	$\frac{1}{N} \sum_{n=1}^N (P_{est,n} - P_{gt,n})^2$
Relative error (RE))	$\frac{ P_{est} - P_{gt} }{P_{gt}} \cdot 100\%$	Root MSE (RMSE)	$\sqrt{\frac{1}{N} \sum_{n=1}^N (P_{est,n} - P_{gt,n})^2}$
		Normalized RMSE (NRMSE)	$\frac{\sqrt{\frac{1}{N} \sum_{n=1}^N (P_{est,n} - P_{gt,n})^2}}{P_{gt,max} - P_{gt,min}}$

circuit, one could calculate the error (E) defined as the difference between the estimated and the ground truth value, or compute the absolute error (AE) as the absolute value of E (although this may mask a model's potential bias). However, this metric is difficult to compare across circuits, as they exhibit different power ranges. Therefore, the percentage error

(PE) or relative error (RE) is more commonly used, as it normalizes the error to a percentage. When power is considered over multiple cycles or across multiple circuits, the individual errors can be calculated. For multiple power values, the mean absolute error (MAE) and the arithmetic mean of all individual absolute error estimates are used. It can also be calculated using the relative errors, yielding the mean average percentage error (MAPE). The mean squared error (MSE) emphasizes larger errors, making it easier to identify models with significant deviations. To have the average error on the same scale as the individual ones, the root mean squared error (RMSE) is used. One can also normalize the RMSE value by the mean or the range of power values to obtain a normalized root mean square error (NRMSE). This normalization enables better comparability across different models operating at different scales.

2.2 Machine Learning for Power Modeling

Machine learning (ML) encompasses various data-driven modeling methods that can be applied to model IC power dissipation. In power modeling, different ML methods can be deployed depending on the complexity of the relationship between input (feature) and output (label) data. Input features can include high-level architectural parameters or gate-level SA, forming a dataset with output labels, e.g., from simulation-based power traces. Figure 2 visualizes different ML methods that can be applied to power modeling.

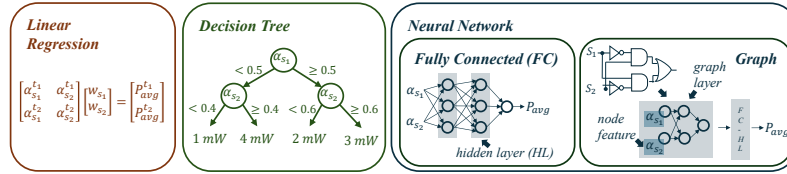


Fig. 2. ML Methods for Dynamic Power Estimation

Regression is used to model the relationship between a dependent variable and one or more independent variables, e.g., for the prediction of a continuous variable. Advanced ML regression methods can be deployed for power modeling, such as *Decision trees* (DT) algorithm, which divides data into subsets based on features and forms a tree-like structure where each node represents a decision rule. Gradient Boosting Trees (GBDTs) are a boosting variant that combines weak models (here, DTs) to form a stronger model with minimized error. *Neural networks* (NNs) consist of layers of interconnected nodes that transform input data to output predictions. Parameters, like the number of hidden layers (i.e. the ones between input and output layer), influence the performance of an NN. Graph Neural Networks (GNNs), as a class of NNs, are designed to operate on graph-structured data, as they appear in circuit design, such as a netlist. Convolutional Neural Networks (CNNs) learn feature relations with filter optimization. For this, the (power) data needs to be converted to an appropriate representation (often 2D but not exclusively). Any ML method can quickly explode in computational effort with higher amounts of training data. Because data can be dependent and correlated, various methods are used to reduce their dimensionality, such as Principal Component Analysis (PCA), L1 (Lasso) or L2 (Ridge) regularization.

3 Review Methodology

We conduct a Systematic Literature Review (SLR) to provide a comprehensive and systematic overview of state-of-the-art studies in the domain of IC power modeling following the SLR procedure presented in [31]. It is defined beforehand and thereby reduces the likelihood of bias in the study selection. It includes the following steps:

Step 1. Specification of research questions (RQs): As we aim to trace the rapid advancement of ML in IC power modeling, report a general shift toward power models at higher abstraction levels, and investigate the open-source power estimation tools, we try to answer the following questions through the proposed SLR:

- RQ1:** What are the benefits and limitations of power models at different abstraction levels, and how do they compare in terms of performance and transferability?
- RQ2:** What are the prospects of using machine learning for power estimation?
- RQ3:** How can we verify the accuracy of a power estimation method, and to what extent does knowledge of the underlying hardware technology influence its accuracy? Is technology-agnostic power estimation feasible?
- RQ4:** What is the extent of open-source power estimation tools? To what degree is power modeling dependent on proprietary data or information from commercial tools?
- RQ5:** What are open challenges in IC power estimation?

Step 2. Development of a review protocol: We design a review protocol to reduce research bias. It declares the databases included in the search, search phrases, study selection criteria, and study quality assessment. The review starts by considering these databases and publishers to identify research and select primary studies: IEEE, ACM, Elsevier, and Springer Link. Each database is searched with all possible combinations of "power model", "power estimation", "power consumption" and "gate level", "RTL", "high level". If title and abstract indicate that a power model was proposed, a study is selected and, additionally, its references and citations of it are checked to find more related papers.

Step 3. Selection and evaluation of primary studies: After considering the review protocol, 173 papers (primary studies) were selected based on title and abstract. The selected studies undergo a quality assessment done by two researchers independently. Each study is evaluated with a focus on methodology and conclusion; the following inclusion criteria were applied:

- I1: The paper is a primary study, published by one of the scientific publishers in the last ten years, and has passed through a peer-review process,
- I2: The main focus of the paper is power estimation/modeling at one of the three abstraction levels: gate-level, RTL, and high-level.
- I3: A power modeling/estimation method was proposed, or an existing method was adapted/optimized.
- I4: A proper verification method was used for model evaluation.
- I5: The power model is not reliant on on-chip performance monitoring counters (PMCs).

Assuming that earlier research is subsumed in works from 2015-2025, I1 allows a stronger focus on current research. I2 emphasizes that we exclude any work where power estimation is not a primary focus, such as a power side channel assessment in the security domain [62] [61]. I3 and I4 are crucial, as the goal of this literature review is to compare different power models and not to compare commercial tools or design flow optimizations. These methods, however, need to be properly verified with ground-truth data. I5 is relevant to our main focus on design-time power estimation methods, whereas PMC-based methods require specific hardware characteristics and need to be modeled separately.

Applying these inclusion criteria reduces the number of selected primary studies and results in a total of 61 primary studies being included.

Step 4. Extraction, synthesis, and analysis of data from selected studies: We designed a data extraction form to collect the relevant data from each study. In addition to title, author names, and publication year, the following aspects are extracted:

- Abstraction level
- Circuit application
- Ground truth data origin
- Power estimation method
- Reference comparison aside from ground truth

Grouping studies by category, we then synthesized and analyzed the extracted data, to answer the RQs.

4 Study Overview

We first categorize the selected studies based on the targeted IC abstraction level, and then we reveal the research trends in IC power modeling in the last decade.

4.1 Taxonomy

Figure 3 presents the top-level categorization overview of the SLR. The primary studies are grouped into three categories: high-level (including the algorithmic level, ESL, and special modeling techniques such as TLM), RTL, and gate-level.

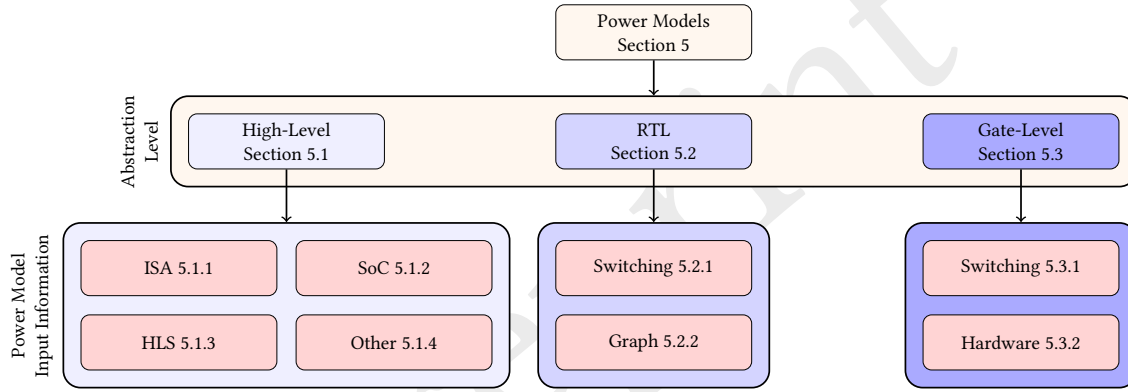


Fig. 3. Top-level overview of the power estimation methods

Here, we sort a study into a dedicated abstraction level if the input parameters of the power model (PM) are based on information from that level. In particular, our categorization relies on the type of information fed to the PM for inference. For instance, if the PM input parameters are from RTL to perform the power dissipation estimation, we sort this PM into the RTL category. It should be noted that we disregard the abstraction level(s) of data that are only used for the creation of the PM (i.e., for training), and we focus on the abstraction level(s) of input parameters used in PM inference. When input parameters originate from multiple abstraction levels, the model is categorized into the lower abstraction level.

Within a design level, the different sources of input information distinguish the PM the most. Hence, the selected studies are further categorized (second-level categorization) based on the type of input parameters of the PM, as follows:

- *Within high-level*: Instruction traces, high-level signal activities of SoC, high-level synthesis (HLS) flow data, other sources (e.g., memory datasheets)
- *Within RTL*: RTL activity simulation, graph-based circuit models
- *Within gate-level*: gate-level activity simulation, data additionally from standard cell libraries or custom gate characterization

4.2 Research Trends

Figure 4a illustrates how many studies fall into which abstraction level. Almost half fall into the high-level category. It, hence, presents the category with the largest number of studies, followed by RTL and then gate-level. System integration of ICs is majorly modeled at higher abstraction levels (e.g., via SystemVerilog or SystemC SoC designs). So, the distribution of the studies shows that the research on power modeling also set a focus on methods that support system integration. In contrast, gate-level estimation was less targeted.

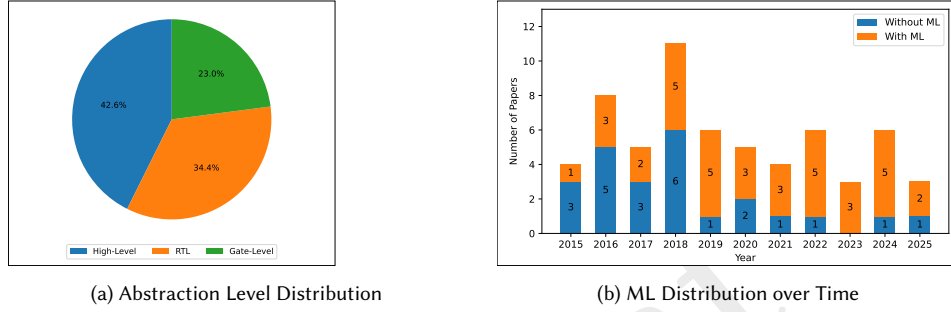


Fig. 4. Distributions of PMs within last Decade

Figure 4b shows the distribution of the selected studies across 2015-2025. The portion of studies that use ML is highlighted in a lighter colour. We observe an increase in ML approaches, and after 2018, they make up more than half of the considered studies. Looking more closely, a variety of methods are used, ranging from non-ML methods, such as analytic-function (Funct.)- and table (LUT)-based ones, to ML methods, like linear regression (LR), decision trees (DT), feed-forward neural networks (NN) and graph neural networks (GNN). A detailed overview of modeling methods is

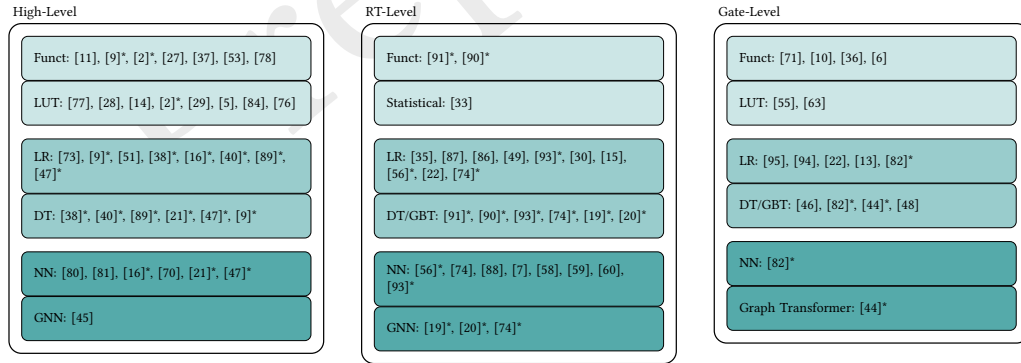


Fig. 5. Overview of the different modeling methods used per abstraction level. A darker color indicates an increase in model complexity. (*) indicates that multiple methods were used or tested)

shown in Figure 5, ordered by the abstraction levels. Methods are grouped into three categories: conventional, ML, NN-ML. Especially at high-level and RTL, ML methods have become a research focus. However, it is also observable that at high- and gate-level, non-ML methods are still considered.

5 Characterization: Power Modeling Approaches

In this section, details of power estimation models at different abstraction levels are presented based on the categorization in Figure 3. Each primary study is evaluated based on the extraction criteria with the goal of answering the research questions. More specifically, we aim to address the following aspects for each abstraction level and summarize the answer as a lesson learned:

- Use Cases (run-time models that serve as on-board power meters or design-time models that estimate power during the design flow, what specific applications are targeted, etc.)
- Types of circuits (CPU, SoC, etc.)
- Type of power model (e.g. ML, LUT, analytical)
- Main research focus (feature selection, transferability, etc.)
- Open-source implementation ratio

We start with high-level methods, which include the majority of works. Following, we discuss RTL and gate-level methods.

5.1 Power Modeling at High-Level

Power modeling at a high level faces various challenges. The primary focus is on providing highly accurate power estimates with a fast design simulator.

For example, in design space exploration, various circuit configurations need to be evaluated. To be versatile, the PM should also be transferable across different IC designs (e.g., CPUs) and technologies (e.g., between ASIC nodes or between different FPGA boards).

Particularly at high levels, only limited information on the circuit design is available (e.g. in early design stages), or it is not feasible to incorporate (e.g., when real-time estimates are required for large designs like CPUs). Hence, these limitations result in a trade-off between achievable accuracy and inference speed of a PM. The available information for power modeling is particularly dependent on the target application of the PM at a high level. This is the primary driving factor for the architecture of a PM at this abstraction level and dominates the choice of modeling method (e.g., LR or NN).

Table 3 summarizes the methods. The clustering shows the PMs' type (in accordance to the second-level categorization in Section 4) and their major parameters, including the modeling method, input parameters, ground truth data for training (if applicable) and evaluation. The PMs available from open-source repositories are marked with a *, highlighting a very rare occurrence of openly available implementations at a high level.

5.1.1 Instruction Set Architecture (ISA)-Centric High-Level PMs. Power models utilizing the trace of executed instructions for a binary program executed on a CPU are proposed in [9], [27], [28], [38], [73], [76], [77] and [84]. For the PM generation, a specific ISA is considered. It uses instruction-centric information, such as the count of executed instructions [76] and the resource utilization per instruction type as input features [28].

Figure 6 shows the general flow for inference of the ISA-centric PMs and special extensions to this flow. For a specific binary program, the PM estimates the cycle-accurate power trace (in [9], [27], [38], [73], [77]) or the average power dissipation (in [28], [76], [84]). A binary program is simulated at a high level to obtain the instruction-centric input features of the PM. An instruction set simulation (ISS), such as Spike [32], can be used as in [76]. Alternatively, an FPGA emulation of ASIC CPU designs can be considered [27].

Table 3. PMs at high-level (* indicates work with open-source repositories)

Work	Modeling method	Target application	Input parameters	Ground truth data	Max./ Avg. error [metric]
[76]	Table	CPU	Instruction	Gate-level power (RTL-based SA)	9.7%/ 4.8% [RE]
[28]	Table	CPU	Microcomponent & processor state	Gate-level report (without SA)	1.2%/ - [RE]
[77]	Table	CPU (SystemC/ TLM libraries)	Instruction, processor state	On-board measurements	6.1%/ 2% [RE]
[84]	Table	CPU (RocketChip)	Instruction, processor state	Gate-level power (RTL-based SA)	2.8%/ 0.8% [RE]
[27]	Analytic	CPU	Instruction, processor state	Gate-level power (with RTL SA)	8.8%/ 4.5% [MAPE]
[9]	Analytic, LR , DT	CPU (branch prediction)	Custom ISS counters	High-level power (with high-level SA)	$\approx 35\%$ / $\approx 5\%$ [RE]
[73]	LR	CPU (TLM)	TLM port SA	On-board measurements	19.4%/ 5.5% [RMSE]
[38]	LR, DT	CPU	High-level signal SA	Gate-level power (with gate-level SA)	34.7%/ 2.2% [per-cycle RE]
[5]	Table	IP Blocks (TLM)	RTL to TLM-mapped SA	Gate-level power (RTL-based SA)	2.2%/ 1.2% [RE]
[14]	Table	SoC (TLM)	Input signal SA	Gate-level power (with RTL SA)	1.7%/ 1.5% [RE]
[2]	Table, analytic	SoC (TLM)	Instruction & TLM port SA	Datasheets & measurements	22.3%/ 12.4% [RE]
[37]	Analytic	IP Blocks	Gate-level netlist structure	Gate-level power (with gate-level SA)	152.9%/ 108.9% [RE]
[16]	LR, NN	SoC	unclear	Gate-level report (without SA)	4.8%/ 2.4% [RE]
[70]	NN	IP Blocks (FPGA)	Control state, input signal SA	On-board measurements	0.8%/ 0.7% [MAPE]
[40]	LR, DT	IP blocks	Signal SA (3 granularities)	Gate-level power (with gate-level SA)	10.1%/ $\approx 6\%$ [per-cycle MAE]
[89]*	LR, DT	CPU	Configuration, SA, McPAT	Gate-level power (with gate-level SA)	7.4%/ 5.5% [MAPE]
[21]	DT , NN	SoC	High-level and RTL parameters	gate-level PPA reports	2.9%/ 2.5% [MAPE]
[29]	Table	HLS (Coarse grained ASIC)	Microcomponent	Gate-level (post-P&R) power	8.1%/ 6.0% [RE]
[53]	Analytic	HLS (FPGA)	LLVM IR	On-board measurements	3.5%/ 1.8% [MAPE]
[80]	NN	HLS (FPGA)	LLVM IR, HLS	Gate-level (post-P&R) report	7.9%/ 4.0% [RE]
[81]	NN	HLS (FPGA)	LLVM IR	Gate-level (post-P&R) report	5.2%/ 2.7% [RE]
[47]	LR, SVM, DT , CNN	HLS (FPGA)	Resource utilization, signal SA	On-board measurements	7.8%/ 4.7% [MAE]
[45]	GNN	HLS (FPGA)	Resource utilization, signal SA	On-board measurements	11.2%/ 3.6% [RE]
[11]	Analytic	Memory sub-systems	SRAM compiler	Gate-level power (with gate-level SA)	13%/ 6.1% [RE]
[78]*	Analytic	DRAM systems	Measurements or datasheets	On-board measurement	-/ 18.8% [RE]
[51]	LR	ALU (GPU)	Instruction (register values)	On-board measurements	≈ 35 pJ/ 7.8 pJ [RMSE]

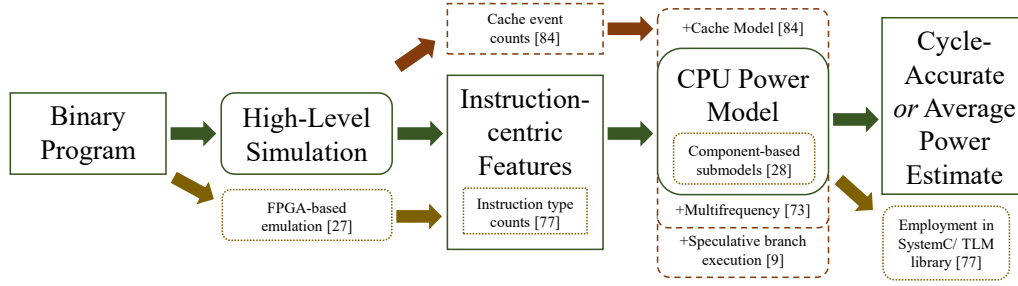


Fig. 6. High-level power estimation inference flow for CPU designs with extensions (dashed) and examples (dotted)

Analytic equations (in [9], [27]), look-up tables (LUT) (in [28], [76], [77], [84]), linear regression (LR) (in [9], [38], [73]) and decision trees (DT) (in [9], [38]) have been investigated for the ISA-centric PMs.

The authors of [27] model the dependency between executed instructions and power, as well as between control states and power (e.g., by using Hamming distance features on signals).

LUTs with average power per cycle data and the average number of cycles for each instruction type are used by [76]. This information is obtained from multiple simulation runs of synthetic instruction-type-specific programs. The resulting simulation files are analyzed by a power profiler. Similar features are used by [84], which also considers cache fills and pipeline stalls as additional event counts for the PM. In [77], the PM is incorporated into SystemC and TLM libraries to extend a virtual prototype of the CPU by a cycle-accurate power profiler. Additionally, the CPU-cache interaction, which can result in pipeline stalls, is modeled using a finite-state machine. In addition to the ISA, microarchitectural information of the CPU was used by [28] by an hierarchical approach based on the CPU's structure and its various resources, such as the ALU. The PM is based on average power values for subsequently executed instruction types per resource, and the resource utilization for each instruction type.

High-level simulation events, e.g., from a SystemC/TLM functional simulation, form the input features for a linear regression (LR) PM in [73]. The model is split into a frequency-independent and a frequency-dependent part, which is calibrated to different clock frequencies to support multifrequency operation of a CPU. The authors of [9] proposed a calibration of a baseline ISS-simulation trace-based PM for speculative branch prediction. Additional cycle counts for incorrectly executed instructions are captured via a custom extended ISS. The PM's input features are these counts, the instruction-per-cycle (IPC) metric and the average total power value obtained from the baseline model. For these features, LR is compared with an analytic equation-based method and a decision tree (DT). Similar to [28], [38] considered a hierarchical CPU model by incorporating microarchitectural information. The CPU-level PM is a superposition of individual PMs per block. Different block types use different input features, such as the Hamming distance between input signals for subsequent instructions in datapath blocks (e.g., the ALU). For the blocks' PMs, different LR setups (e.g., using L2-norm regularization) were compared with DT and gradient boosting decision trees (GBDT).

Evaluation: Various ISAs and microarchitectures have been considered for verification of the PMs, especially RISC-V (in [38], [76], and [84]). In addition, ARM ([73]), FPGA-based softprocessors ([77], [27]) and OpenRISC ([9]) were used. Different benchmark programs (ranging in number from 5 to 34) were considered in the evaluation, with little to no overlap in their selection.

An overview of the maximum and average errors reported for the PMs is shown in Table 3. The methods utilized various ground truth sources, including gate-level simulation data ([27], [28], [76]) and on-chip measurements ([77]).

Most methods have compared their PM only to the ground-truth power data. However, [9] and [38] compared different modeling methods for their input features. In both works, the DT showed high accuracy scores (in [9] the LR was slightly better, which may be caused by insufficient input features according to the authors). In [38], also the impact of the composition of submodule-PMs has been evaluated. The MAE of the composed model is 2.2%, twice that of a single DT model for the entire CPU.

Given the crucial role of execution speed for PMs, table-based approaches remain advantageous. Their simplicity enables large speedups compared to simulation-based power estimation. The authors of [76] report a speedup of 3.5 orders compared to RTL, while [28] achieves a speedup of up to 5 orders compared to gate-level.

In summary, ISA-centric PMs were refined over the last decade to support specialized phenomena, such as speculative branch prediction. Especially simple modeling approaches, such as tables and LR, remained in focus. While the methods have been evaluated on current ISAs, such as RISC-V, the PM's transferability across ISAs has not yet been focused on.

5.1.2 System-on-Chip (SoC) and Intellectual Property (IP) Block-targeted PMs. PMs for entire SoCs, consisting of multiple IP blocks, were proposed by [2], [5], [14], [16], [21], [37], [40], [70] and [89]. They focused, for example, on mapping lower-level information to high-level IC design views (e.g., RTL signals to TLM ports) and extending simulators with cycle-accurate PMs.

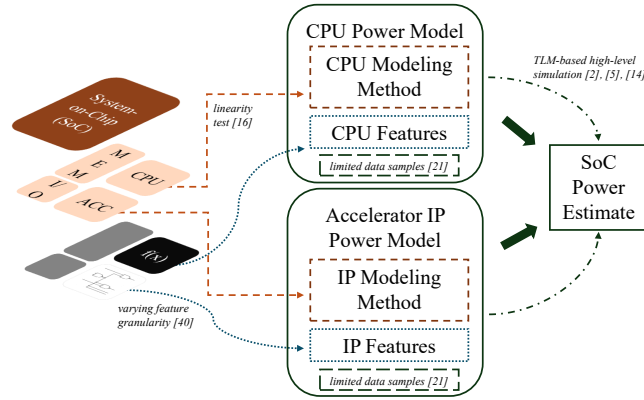


Fig. 7. Power modeling of SoCs with IP blocks utilizing additional features and processes

Besides this signal mapping, the (IP) block-based structure of SoCs builds the basis of the models. Figure 7 shows the concept of individual PMs per block with block-specific input features and modeling methods. For example, an existing ISA-centric PM can be used for the CPU, whereas an accelerator IP block requires a custom switching-activity-based PM (as done in [2]). The models are incorporated into high-level simulations to provide cycle-accurate power estimates (in [40], [2], [14], [5], [37]). However, PMs were also proposed that target average power estimates (in [16], [70], [21], [89]).

High-level simulations are extended with PMs in [5], [2], [14] and [37]. Similar to ISA-centric PMs, simple analytic-equation-based (in [2], [37]) or table-based (in [2], [5], [14]) methods are used. In [37], a net-centric analytic equation is defined for IP blocks. Average power values obtained from gate-level power estimates in sample simulations are combined with workload statistics. The IP block's netlist is translated into a C module for high-level simulation, which uses the net-centric analytic equation to estimate power. In contrast, [2] incorporates the PM into TLM, focusing on multi-processor SoCs. For the CPU cores, an ISA-centric, table-based PM is incorporated in a TLM wrapper. For

peripheral blocks of the SoC, a TLM module employs an analytic equation, based on the signal activity of the primary inputs and outputs. In addition, [5] and [14] consider the mapping of internal RTL signals to TLM ports.

Regression-based PMs ([16], [40], [70], [21], [89]) were investigated for block-specific modeling of SoCs. A feed-forward neural network is proposed by [70] to model IP blocks with a similar structure (i.e., same number of input and output ports) but different combinatorial datapaths. The PM uses the switching activity of data signals and the state of control signals. The authors of [16] use a linearity test to identify whether an LR or a feed-forward NN is more suitable for a block's PM. For the NN-based PMs, a genetic algorithm is leveraged to find suitable network and training parameters. Most methods focus solely on switching activity-based input features that influence dynamic power. In [21], individual PMs for both dynamic and static power are considered (with a focus on a limited number of power samples per IP design). Various high-level features (e.g., cache configuration) of an IP block are combined with parameters such as the clock cycle length. For the feature set, PMs based on feed-forward NN, GBDT, Gaussian process regressors, PR and DT have been compared. Similarly, [40] considered various regression methods (namely, LR, DT and GBDT) for an IP block-centric PM. Activity features are obtained from annotated high-level simulations, considering three different granularities (i.e., using all internal signals, only input and output signals of an HLS tool's block-based IR, or solely input and output signals of the entire IP block). Ensemble learning was used for more robust estimates of the IP block's input-output-based model. In [89], the focus is set on a regression-based PM (comparing LR, DT and GBDTs) for CPUs using CPU configuration parameters, high-level activity features from simulation and a power estimate from the hierarchically structured, analytic equation-based McPAT [42] PM. The samples in the dataset are identified by an active learning approach and a custom multicollinearity-detection algorithm based on the variance inflation factor reduces the number of features.

Evaluation: Various benchmark circuits have been considered for the SoC-targeted PMs, including custom IP designs [70], simple adders and multipliers [37] or timer IPs [14]. But also IP blocks, synthesized by HLS [40], and CNN accelerator and SRAM configurations [21] were used for evaluation. Furthermore, SoCs with ARM CPUs and CGRAs [16] or RocketCore [4] and BOOM [8] were considered.

Similarly, the ground-truth origin of power data is diverse:

- Gate-level power reports without simulation (in [21])
- Gate-level-based power estimates with RTL simulation data (in [14], [5])
- Gate-level simulation-based power estimates (in [16], [37], [40], [89])
- On-chip measurements (in [2], [70])

Individual error results are collected in Table 3. The PMs were mostly compared only to the ground-truth power data. However, it is observable that the average error of 12.4% reported in [2] is 10 times larger than that of [5], although both methods target a high-level power estimate with TLM-based simulations. This could be based on different ground-truth origins (gate-level vs. on-board measurements), which complicates the comparison. In [89], the PM has been compared to power estimation methods, based on on-chip performance counters (e.g., [41], [83]). Here, the proposed PM showed, on average, a 4-fold improvement in accuracy. Furthermore, using active learning on a smaller dataset (70% of the total number of samples) had only a small effect on the PM (on average, accuracy increased by 0.25%).

In particular, [37] reports large error values of 108.9%. However, the authors claim a more than 7 times higher accuracy than using an average power value over the entire simulation time. Additionally, the focus was on achieving fast simulation times (with a speedup of up to 4 orders of magnitude compared to a commercial toolchain) and on simple integration of an IP block into high-level system simulation.

The authors of [16] evaluated a linearity test as additional guidance on the model choice. It showed, that for a CPU and a CGRA block, the test identified the correct direction.

Different modeling methods were compared by [40], [21] and [89] for their input feature sets. Here, the DT and GBDT-based models outperformed approaches such as LR and feed-forward neural networks. However, [21] also utilized information from generated RTL designs of the benchmark circuits, and could therefore be considered a cross-level approach.

In conclusion, for SoC designs composed of multiple IP blocks, it is observable that decomposing the PM according to the system’s hierarchy enables selecting individual features and modeling methods. While ML-based models were considered, simple analytic equation- or table-based models remain the focus for cycle-accurate power estimation in high-level simulations. Here, power estimates for different workload applications were considered almost exclusively, but transferability to, e.g., other FPGA platforms was not in focus. Only [89] additionally considered CPUs from the same framework with different configurations.

5.1.3 High-Level Synthesis (HLS)-targeted PMs. Power estimation prior to or during high-level synthesis (HLS) constitutes a distinct sub-domain of power modeling for SoCs and IP blocks and has been addressed in several works [29], [45], [47], [53], [80], [81]. Figure 8 shows the different information sources that are used for the PMs. For a C/C++ algorithm intended for hardware deployment, an important objective is to estimate power before performing time-consuming HLS (e.g., for early-stage design space exploration). To this end, information from the LLVM [39] IR was utilized by [53], [80] and [81]. Another key objective is the accuracy of power estimates from the HLS stage. Significant errors

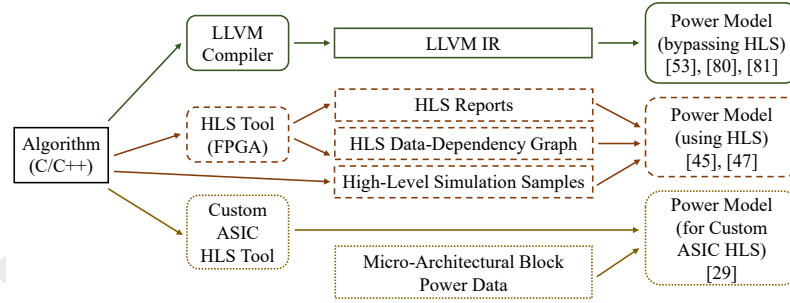


Fig. 8. Information sources for HLS-targeted PMs

between a gate-level and the HLS tool’s average power estimate have been observed by [29]. To overcome this gap, the authors of [45] and [47] aim to improve PM’s accuracy by utilizing various FPGA-targeting HLS information. A PM for a framework of predefined micro-architectural block-based design, SiLago [26], and a dedicated ASIC-HLS tool Sylva [43], is proposed by [29].

In [53], an analytic-equation-based PM is proposed that utilizes information from the LLVM IR, like operator counts and levels of nested loops. The different operator counts are weighted by empirically found constant factors. In an HLS process, directives such as loop unrolling influence the resulting RTL design. This impact is accounted for by weighting the PM’s input features further. In contrast, [80] and [81] use only the count of certain operators in the LLVM IR. These counts serve as input features to a feed-forward NN with one hidden layer. DSP instance counts from the HLS tool are used as an additional input feature to refine the power estimate by [80]. In [81], the PM is refined with more operator types and more nodes at the NN’s hidden layer.

To extend their custom ASIC HLS tool, [29] builds their PM on two components. A per-microarchitectural block table-based PM is generated from power data using simulation with synthetic testbenches. An analytic-equation-based model captures the interdependencies between multiple blocks.

To refine power estimation after HLS, [47] extracts architecture features, such as FPGA resource utilization, from the HLS tool's report. These features are complemented by switching activity. The activity information is collected via a high-level simulation, in which RTL operator signals are mapped back to the C/C++ algorithm. Various regression methods, including LR, support vector machines, DTs and NNs, were investigated. A GNN, leveraging the intermediate graph representations in HLS processes (i.e., the RTL operator connectivity), is proposed by [45]. The operators are used to form the GNN nodes, while switching activity information is obtained similarly to [47] and used as edge features. A custom edge-centric GNN layer aggregates the edge information a feed-forward NN layer processes the architectural features.

Evaluation: In contrast to other applications of PM at high-level, HLS-targeted methods were evaluated with similar C/C++ algorithms from MachSuite [69] (in [80], [81]), CHStone [25] (in [53], [80], [81]) and Polybench [65] (in [45], [47], [53]) ([29] used custom benchmarks).

The HLS-targeted PMs were evaluated against power estimates from post-P&R stage ([29], [80], [81]) and, especially for the FPGA-targeted HLS flows, against on-board measurements ([45], [47], [53]). Individual error results are shown in Table 3.

It can be observed that [53] shows significantly better results than [45] and [47], although both methods use similar benchmark suites and ground truth sources. This is likely due to extensive domain knowledge and empirical observations on an HLS tool that were manually incorporated into the analytic equations in [53]. In contrast, [45] and [47] automatically extracted various features from the HLS tool and applied them to advanced regression methods. These results suggest that domain knowledge has a stronger influence on the PM's accuracy than the underlying modeling method.

Different regression methods were evaluated by [45], [47] and [81] for their respective feature set. More advanced methods, like larger NN architectures and custom GNN layers, lead to improved accuracy. For instance, increasing the number of hidden nodes in the NN architecture between [80] and [81] resulted in a sixfold increase in accuracy. The custom GNN architecture in [45] achieves better results than standard GNN layers, such as GraphSAGE [24], and slightly outperforms the CNN of [47].

Overall, the customization for a specific target platform and HLS tool yields higher accuracy gains than an advanced modeling method for HLS-targeted PMs. Consequently, transferability is only marginally addressed by the proposed PMs. Notably, only [53] evaluated their model on a different FPGA platform and reported a threefold increase in error, indicating that additional modifications would be required to maintain the PM's accuracy across technologies.

5.1.4 High-Level PMs for Other Applications. Memory power estimation is crucial for early design stage exploration in SoC designs. Average power estimates for large memory subsystems of SRAM instances are considered by [11]. Cycle-accurate power estimates that are incorporated by libraries (for various standards) into a high-level DRAM subsystem simulator are in focus of [78]. Due to the proprietary nature of memory architectures and data, primarily datasheet values (especially for recent memory standards) are available as ground-truth data. Accordingly, both [11] and [78] proposed analytic-equation-based PMs. For the SRAM-based memory subsystem, [11] derived analytic equations for the memory internal power (i.e., the SRAM instances), the clock network power, the interconnects power and the internal power of flip flops. In [78], the PM is divided into one for the DRAM core itself and one for the interface to the

memory controller. Both PMs are evaluated on individual benchmark systems and error information can be found in Table 3.

[51] focuses on average power estimation for ALUs within graphics processing units (GPUs). LR is used, with the Hamming distances of the input and output values of two subsequently executed instructions serving as input features. As there is no direct access to the ALU in an industry-level GPU, an indirect measurement of ALU power is used for training data generation. The accuracy is shown in Table 3. While only absolute energy values have been provided, the method is claimed to be over 80% more accurate than a data-independent instruction-based model of the GPU.

There exist studies on PMs at high-level that fall out of the scope of this SLR, either as they focus specifically on still-niche circuit designs, like 3D stacked CMOS image sensors [79] or target the software domain above the hardware stack [12, 68].

Lesson Learned from High-Level PMs:

Use-Case: Most PMs have been proposed for evaluating IC designs at design time. High-level power modeling targets specific application scenarios and leverages their specific features (like the instruction types of ISAs). This specialization is often necessary due to limited information (e.g., missing detailed switching activity from internal signals), which would allow for generally applicable models.

Types of Circuits: While most methods target CPUs, also individual IP blocks and SoCs are in focus.

Types of Models: Although table- or analytic-equation-based models remained in focus, ML-based ones have received attention in recent years. Incorporating application-specific domain knowledge and the specialization to single circuit types enables accuracy improvements even for simple modeling methods.

Research Focus: Accuracy is mainly improved by focusing on information sources for special phenomena, like speculative branch prediction, which limits the transferability of the PMs to other circuits. An improvement in this direction is targeted for system designs by a hierarchical decomposition into sub-PMs for the system blocks. While the estimation speed is (implicitly) targeted, e.g. by the choice of fast table-based methods, it is explicitly evaluated seldom.

Open-Source PMs: Only two studies have provided open-source repositories.

5.2 Register Transfer Level

The majority of power estimation models at RTL focus only on dynamic power since switching activity is the biggest hardware independent variable. Static power, however, is highly technology dependent and not directly influenced by switching. Switching activity can be observed at all levels with varying degrees of granularity. An overview of the different approaches is given in Table 4. The clustering represents the subsequent subsections where each model is presented and a PMs major parameters, including the modeling method, input parameters, and ground truth data for training (if applicable) and evaluation.

5.2.1 Switching Activity-Based Power Models. Switching activity is a major variable factor of dynamic power and therefore often used as primary input to power models. [33], [58], [93], [59], [60], [72], [91], [49], [7], [90] focus on power modeling at design time while [30], [56], [15], [35], [87], [86], [22] focus on run-time power estimation. At the basis of all models is the RTL switching activity. The amount of RTL signals increases with design size which causes an increase in modeling complexity. Since RTL signals often correlate with each other, the total signal amount can be minimized. Methods to reduce the number of signals are:

Table 4. PMs at RTL (* indicates work with open-source repositories)

Work	Modeling Method	Target Design	Input Parameters	Ground truth data	Maximal Error [Metric]
[30]	LR	IP Blocks	RTL simulation	gate-level power	3.35% [NRMSE]
[33]*	Sampling	CPU	RTL simulation	gate-level power	2% [RE]
[56]	LR, MARS, NN	CPU	RTL simulation	post-P&R gate-level power	3.62%, 2.92%, 4.02% [RE]
[58]	MLP	adder, multiplier	RTL simulation	gate-level power	7.4% [MAPE]
[15]	LR	FIR filter, arithmetic toolbox	signal activity monitors	On-board measurement	9.8mW [AE]
[35]*	LR	CPU	RTL simulation	gate-level power	9% [NRMSE]
[93]	LR, GBT, MLP, CNN	CPU, circuits	RTL simulation	gate-level power	5.2% [NRMSE]
[59], [60]	NN	circuits	RTL simulation	gate-level power	1.35% [RMSE]
[72]	LUT	CPU	RTL simulation	post-P&R power	9.6% [MAPE]
[87]	LR	CPU	RTL simulation	RTL power	10% [NRMSE]
[86]*	LR	CPU	RTL simulation	gate-level power	10% [MAE]
[91]*	Analytical, GBT	CPU	RTL simulation + architecture info	gate-level power	8% [MAPE]
[22]	LR	Accelerators	RTL simulation	hardware measurements	5% [RMSE]
[49]*	LR	CPU	RTL simulation	gate-level power	2.5% [MAPE]
[7]	MLP	CPU	RTL simulation	gate-level power	3.8% , 17.8% [NRMSE]
[90]*	Analytical, GBT	CPU	RTL simulation	gate-level power	5.8% [MAPE]
[88]	MLP, Transformer	CPU	RTL simulation, circuit graph	gate-level power	0.6 [RSSE]
[74]	LR, DT, GBT, NN, GNN	CPU	RTL simulation, circuit graph	post-P&R gate-level power	0.95 [R ²]
[19]*, [20]	GCN, GBT	circuits	RTL simulation, circuit graph	gate-level power	0.54 [RSSE]

- Singular value decomposition [30], [35]
- Principal component analysis [93] (LR, GBT), [49]
- K-means clustering [35], [49]
- Minimax concave penalty [87], [86]
- LASSO + correlation-based weight pruning [7]
- correlation feature selection, correlation subset evaluation, genetic algorithm [56]
- iterative algorithm [22]

[35] proposes simulated annealing to find the optimal number of clusters for the k-means clustering algorithm. [86] considers signals bit-wise instead of complete signals and minimize the number of relevant bits over all signals. [87] and [49] experiment with different numbers of selected signals. [87] proposes using a genetic algorithm to create a variety of micro-benchmarks for training. The final test set is made up of early (random) benchmarks with low power consumption and later benchmarks with high power consumption. [7] proposes a two-step reduction: building on top of [64] LASSO is used for initial signal reduction method and signals are further sparsified with pruning and used to train an MLP. [58], [59], [60] only use average switching activity. [22] proposes an iterative algorithm that only adds

signals if they increase the model accuracy with consideration of the hardware overhead. The (reduced) signals are then input to the following models:

- LUT [72]
- sample based [33], [49]
- LR [30], [56], [15], [35], [93], [87], [86], [49], [22]
- Gradient Boosting Tree [93], [91], [90]
- MARS [56]
- MLP/ANN [56], [58], [93]*, [59], [60], [7]
- CNN [93]

[33] obtains random samples from RTL simulation, simulates power traces for each and uses reservoir sampling to build a power model. [72] partitions the design into regions based on a pre-defined maximum hierarchy depth. Regions consist of gates and RTL signals. For each region a power model is created. A classifier is proposed that groups transitions into different switching behaviors; for each region all transitions of the same group are averaged to form the power value. Total power is the sum of the power of all regions.

For the CNN model in [93], the switching activity of a register or signal is represented in 2D with three channels; the channels serve to encode the switching pattern. The connectivity in 2D is based on the trace file. Different graph based methods were tested to achieve higher local information but results only improved slightly. [93] compares LR, GBT, ANN, CNN and [56] compares LR, MARS, ANN. [91] and [90] use an additional analytical function based on architectural characteristics to scale the results of the ML model.

[58], [59] and [60] propose to estimate power of a component by summing the power of its sub components. This requires full characterization of a set of components. Therefore, two ANNs are trained: one for power estimation and one for activity propagation between sub components. The only difference between the two ANNs is the output layer which is one for the power model (dynamic power) and two for the propagation model (average activity and static probability). [60] introduces a model adaptation of [59] that considers glitches by accounting for multiple switches of a signal in one observation window. Run-time methods [30], [56], [15], [35], [87], [86] include hardware implementations of their models. Due to space and performance limitations in hardware and time constraints [30] and [86] quantize the weight vector. [87] compares their model in software with a quantized version. [15] presents an online power model that uses recursive least squares to update the model with new samples at run-time.

Additionally, transferability of power models was explored in the following ways:

- Different clock frequencies [60] by adding a frequency input to the ANNs
- Different technologies [91], [90] by training a separate ML model
- Different designs [49] by identifying similar signals as samples and retraining the model

[33], [35], [86], [91], [49] and [90] provide open-source code of their power model.

Evaluation: Table 4 shows an overview of the highest reported error. The accuracy of the power model is measured using ground truth data from gate-level power [58] (average), [59] (average), [35], [93], [50], [49], [7], RTL power [87], post-P&R gate-level power [33] (average), [56] (average), [72], [86], and on-board measurements [22]. The models are tested on different types of circuits: RISC-V CPU [93], [72], BOOM [33], [91], [49], [7], [90], CPU [33] (Rocket), [86] (commercial), [35] (Rocket+Hwacha), [87] (Neoverse N1, Cortex-A77), SoC [56] (SecretBlaze), generic benchmark circuits/IP blocks, [30], [93], [58], [59], [60]. In [93] the CNN model achieves the highest accuracy over all benchmarks compared to LR, GBT, ANN. In [56] MARS performs best and NN worst. This is, however, directly dependent on the final reduction method which is a combination of CFS and CSE. When observing the selected signals most were related to data or address buses.

[87] compares their results with models [64], [93] and [35]. In their experiments, their model outperforms the linear regression model in [93] when the number of selected signals exceeds a threshold of total number of RTL signals. They always outperform [64] and [35]. [86] compares model accuracy and hardware area overhead to [87], [35], [64], [95]. The hardware area overhead in [86] is smaller than all while achieving similar or better accuracy. [7] compares the results to [30], [64], the CNN model in [93], and [87]. [7] outperforms all models but the CNN model. However, it is faster and requires less training timing and overall computing resources. [60] shows better results for all components when compared to a linear regression model. The addition of the glitch model in [60] shows direct improvement in the maximum relative error (5.5% compared to 8.5% in [59]). The clock frequency transferability model [60] is tested with three different frequencies ranging from 200-500MHz with a power model trained on 100MHz power data. Relative error increases with frequency increase but for the tested scenarios stayed below 8.4%. [91] and [90] test the model on BOOM CPUs with different architecture parameters and [90] also compares to a different BOOM architecture.

Linear regression methods dominate PMs at RTL as they are relatively low on complexity and do not require extensive training (compared to NNs). Due to their light weight nature they are especially popular with on-board run-time applications that provide power estimates during circuit execution. ANNs, however, often show better overall accuracy especially due to the non-linear modeling capabilities. The better accuracy comes with the tradeoff of requiring more training data and computational resources. Gradient boosting trees seem to fall somewhere in between LR and ANN with regards to accuracy (but may potentially use more computational resources).

5.2.2 Graph-based Power Models. RTL descriptions can be converted into graph representations to leverage graph properties such as propagation and connectivity. Graph-based methods are used for power estimation in [88], [74], [19], [20]. [88] and [20] predict area, power and timing, [74] predicts negative slack and dynamic power. In [20], [19] is extended to account for distinct technology libraries and proposes how to generate RTL designs for training. RTL can be converted into graphs with Verilator [74] or Yosys [88] via the respective intermediate representations. The different graph representations used are abstract syntax tree (AST) [88], [74] and simple operator graph (SOG) [19], [20]. Both are directed graphs between input and output ports. Operations in between are translated into a subset of simple logic operations and multi-bit signals are converted into logic bits. The number and type of operations differ in each method. Each node is annotated with a toggle rate. [88] includes the bit width of each functional block as a characteristic for word-level operators while [19] and [20] consider bit-level operators.

[88] samples specific paths over the entire graph and considers them individually instead of considering the complete graph. The number is restricted to only include paths that begin or end with an input/output port or a register. This is done to capture one cycle behavior and reduce the search space. A depth-first algorithm is used for sampling. The sampled paths are inputs to a Transformer variant which yields the physical characteristics of each path. An MLP is used to aggregate the information of all paths into estimates for area, power and timing.

[74] explores vector based feature extraction as well as two types of graph feature extraction. Models tested are linear regression, random forest regression, NN, XGBoost for the vector representation and GNN for the graph representation. For vector based feature extraction, the AST is back traced for each register and a logic tree of all driving components is created. Features, such as input/output bits and total operator bits, are extracted out of all logic trees into a vector. Additionally, a 2D vector consisting of average tree depth and average wire width over all trees is created. For comparison, two graph representations were also tested, AST graph and a control and data-flow graph.

[19] and [20] support both vector-based and vector-less power simulation but trains the model with vector-less power. Switching activity is propagated by traversing the graph according to each operator type. A graph convolutional

network and gradient boosting trees were tested. After model training, another tree based model was used to calibrate the power prediction. [20] presents a transfer model for different technologies. The difference between target and source library is evaluated based on key features and the model is retrained with only one sample from the new library. Additionally, two types of training data augmentation are presented: graph-based with graph generative models and LLM-based.

Evaluation: [88] generates datasets for training from hardware designs such as Chipyard and NVDLA. [74] uses data generated from 51 RTL designs from IWLS 2005 benchmarks and opencores with post-placement power values as ground truth. Overall, gradient boosting trees perform best with a correlation coefficient of 0.95, while the graph versions showed poor performance and a generally more narrow spread of predicted values. [19] uses 90 different open source RTL designs from different benchmark sources for training and validation. The model is trained vector-less with gate-level power as ground-truth. Results are compared with [88] and [74]. [19] claims the following advantages of SOG over AST: higher similarity to netlist regarding consistency in register mapping and overall a more general representation that reduces difference between different RTL design types and styles. To support their claim, the models proposed in [88] and [74] were executed with SOG instead of AST. Estimation results showed higher accuracy with SOG.

Graph-based methods represent a new branch of power estimation with a lot of potential to bridge the gap between RTL and gate-level. The instrumentation of existing tools such as Yosys and Verilator facilitate the creation of graphs without requiring complete synthesis. Even first attempts at a transferable model have been made. There is still more research that needs to be done in terms of versatility and restrictions and in direct comparison with other ML methods.

Lesson Learned from PMs at RTL:

Use-Case: PMs at this level are used almost equally for design-time and for run-time estimation. Input features include almost always switching activity, while additional information is seldom used. Ground-truth is mostly gate-level power in an overall attempt to train models that are able to give accurate power estimates before synthesis

Types of Circuits: Most of the power models are designed for CPUs with a few works utilizing smaller circuits.

Types of Models: The modeling methods at RTL utilize different ML techniques. Run-time models most often use linear regression while design-time models also include NNs. Graph-based models in particular experiment with a variety of models and model combinations.

Research Focus: Various different reduction methods are explored to achieve better performance and facilitate implementations in hardware for run-time models. Alternative representations, such as graphs, are explored and might be a valid alternative to more traditional approaches, depending on the use case. A few transferability methods are proposed with good results, but more research is necessary, especially when working with trained ML models. At RTL, there is a lot of cross-referencing between methods and direct comparison with previous approaches, which facilitates overall comparison between modeling methods. However, often the use cases between the methods that are compared differ, and different ground truths might have been used. Additionally, the choice of error measurement might differ and distort direct comparisons.

Open-Source PMs: Only a third of works provided open-source repositories of their power model.

5.3 Gate-level

At gate-level the technology is determined and hardware characteristics such as capacitances and voltages can be directly included in a PM. Additionally, the switching activity has a finer granularity which generally provides switching-based models with higher accuracy compared to switching-based models at higher levels. The granularity comes with the caveat of a higher data density and potentially longer processing. Applications vary from run-time to design-time modeling; models vary from purely switching activity-based to methods that use the additional hardware information. An overview of the different approaches is given in Table 5. The clustering represents the subsequent subsections where each model is presented and a PMs major parameters, including the modeling method, input parameters, and ground truth data for training (if applicable) and evaluation.

Table 5. PMs at gate-level

Work	Modeling Method	Target Design	Input Parameters	Ground truth data	Maximal Error [Metric]
[46]	DT	LUT/DSP-based benchmarks	gate-level simulation	post-P&R gate-level power	5.8% [MAE]
[64]	LR	FIR,AES, RI5CY	gate-level simulation	post-P&R gate-level power	4.3%(dyn), 16%(stat) [MAE]
[95]	LR	SoC	gate-level simulation	avg. post-P&R gate-level power	12% [MAPE]
[94]	LR	CPU (SIMD)	gate-level simulation	avg. post-P&R gate-level power	1.8% [RMSE]
[13]	LR	SoC	gate-level simulation	avg. post-P&R gate-level power	5% [RMSE]
[10]	Analytical	FSM	liberty file, input signals	On-board measurement	22% [RE]
[36], [71]	Analytical	FSM, 5-ring oscillator	liberty file,input signals	On-board measurement	20% [RE]
[48]	Forest regression	ISCAS85, DesignWare	liberty file,input signals	gate-level power	8.6% [RE]
[6]	Analytical	ISCAS85, AxBench	liberty file,input signals	gate-level power	2.14%[RSE]
[63]	LUT	ISCAS85, IS-CAS89	custom characterization, input signals	transistor-level power	10.8% [NRMSE] (inst.), 13% [MAE] (avg.)
[55]	LUT	ISCAS85	custom characterization, input signals	transistor-level power	12.6% [RE]
[82]	LR, GBT, ANN	gates (2-5 inputs)	input signals	transistor-level power	3.98% [NRMSE]
[44]	GBT, Graph Transformer	CPU	gate-level netlist, liberty file, input signals	post-P&R gate level	0.8% [MAPE]

5.3.1 Switching Activity-Based Power Models. Run-time dynamic power models are created at gate-level with decision trees [46], and linear regression [64], [95], [94], [13]. The decision tree in [46] is implemented in hardware memory. Hyper-parameters of the decision tree such as depth are determined empirically and with k-fold cross validation. Additionally, an ensemble strategy is proposed; each component or function is modeled by a decision tree and all are summed up to yield the estimate of the whole design. This facilitates reusing models in new compositions. [64] proposes a dynamic and a static power model. With an empirical study they conclude that the biggest influences to

static power are temperature and different voltages. The static power model then consists of a polynomial regression to account for the non-linear relationship between those two parameters. [94] considers multi-threaded architectures and [13] focuses on hardware allocation to consider resource constrained power monitors on FPGAs. Dynamic power can be estimated by monitoring the switching activity of a small subset of significant components. Input to all models is switching activity. A lot of gates correlate strongly due to the circuit topology.

To limit the amount of hardware counters different signal reduction mechanisms are used:

- Signals with highest switching activity are selected and redundant signals are filtered out [46]
- Regularized linear regression model (Lasso). As this model is sparse, coefficients can be ordered and only signals with high correlation are selected [64]
- Iterative addition of sub-modules with custom algorithm [95], [94], [13]
- Multicollinearity analysis to group signals with similar switching activity together and only keep one signal per group [13]

[95], [94] and [13] create a power model in an iterative top-down algorithm that considers both prediction error in terms of accuracy (relative error) and an estimated cost value in terms of area and power overhead. The power models take switching activity information of data signals as input obtained from post-synthesis VCDs. Control signals are added as inputs iteratively only if the addition improves accuracy. Iteratively the algorithm checks accuracy and cost and if results are not sufficient, power models are created for the modules on the next lower level. The power model is a linear regression model trained via a bootstrap re-sampling algorithm where benchmarks are selected randomly for each iteration. The iterative algorithm is extended in [94] by identifying signals used by multiple models to reduce total signal size. In [13] the modeling algorithm favors model compilations with fewer signals based on the resource constraints.

Evaluation: The ground truths used are: post-P&R gate level simulation power traces [64], [46], and average post-P&R gate level simulation power traces [95], [94], [13]. To get more training data from average power simulations, the simulation is split into 100 cycle long sections and power traces created for each. The final power trace is then a series of power samples. Ground truth for static power [64] is SPICE simulation to account for more corner cases than defined in the liberty file. The models are tested on: FIR, AES, RI5CY core [64] (dynamic), ring oscillator as a benchmark(static), LUT-based and DSP-based benchmarks [46], SoC (includes RISCv) [95], [94], [13]. [46] compares results with [15]. In a test with all benchmarks combined, the ensemble variant performs only slightly worse than a re-trained model of the whole design. [94] creates training benchmarks as subsets of the ISA to isolate parts of the design and reduce computational efforts. Here, three partitions were classified: ALU, FPU, LSU. Additionally, instructions are divided into scalar and SIMD instructions. Instructions are executed with random values. The results from the mathematical model and the RTL implementation are compared as well as single-thread and multi-thread instructions. The model was trained only on single-thread instructions, however. The design was also tested for different clock frequencies.

Run-time power models rely on methods with little computational effort in order to be incorporated into the hardware design. The trade-off between model accuracy, hardware area and power overhead are defining design restrictions. Most common approach is LR. Use-cases differ in complexity, especially when compared to run-time methods at RTL.

5.3.2 Hardware Parameter-based Power Models. Many methods use information directly from the target technology either through the characterization in standard cell libraries [10], [36], [71], [48], [6], [44] or through custom characterization with transistor level simulation [63], [55], [82]. One advantage of custom characterization is the potential to account for multi-input events which are often neglected in standard cell libraries.

Aside from switching activity, the following hardware or timing parameters were used:

- load capacitance [10], [36], [71], [48], [6], [44], [63], [55], [82]
- input transition time (slew) [48], [6], [63], [55], [82]
- arrival time (skew) [63], [55], [82]
- pulse width [55], [6]
- number of driving and driven gates [48]
- voltage(s) [10], [36], [71], [63]

Hardware information can be directly included in the hardware description. [10], [36] and [71] propose an analytical model for static and dynamic power which is directly incorporated into the HDL code of the design via a custom library extension. It consists of logic gate models from different CMOS families with embedded static and dynamic power and area estimators. Activity monitors count the switching activity of input/output pins. [6] proposes a one pass simulation flow which includes timing characterization, logic simulation and power estimation. The gate models are created in SystemC which include LUTs from the technology library. A gate model consists of concurrent threads for delay, power, output values and transition time calculations. Transition time and delay propagation estimation is performed with graph-based STA. [63] maps transistor-level simulation results into timing and power LUTs. The gates are simulated with varying parameter values and ranges. The structural information of each gate (driving cells and cells driven) is extracted. The initial input transition is defined and the corresponding output read from the timing LUT. Based on timing combinations the current LUT for each gate is read. This yields the total current profile which is used to calculate instantaneous power and average power. Gate-level models also make use of graph representations while leveraging hardware parameters. [44] proposes a graph-based model to estimate post-P&R power with gate-level information. A design is split into non overlapping modules which are translated into directed graphs. Graph transformer is used as an encoder and is trained to recognize netlist structure, netlist functionality and learn the alignment between gate-level netlist and post-P&R netlist. One special aspect is learning with multiple netlist transformations of the same design with the same functionality. The goal is to train the model to minimize the embedding distance of netlists with the same sub-modules and the same functionality. This is followed by a fine-tuning step with gradient boosting trees. Total power is divided into clock tree power, combinational logic power and register power. The latter two use additional hardware information such as total cell internal power and total cell capacitance from a liberty file.

Imbalanced signal timing may lead to glitches which in turn increase total power consumption. [55] proposes a waveform LUT that considers glitches and their power consumption. Events are grouped together if they occur within a gate-specific maximum interval; interdependencies between events outside of this interval are negligible in terms of glitches. To reduce the characterization cost, an upper limit is set to represent the maximum number of signals that are expected to switch at the same time. The remaining combinations are then simulated analogous to conventional library characterization. [48] presents a regression approach to estimate glitch power. For each gate, parameters such as gate state switching time, output load, number of driving and driven gates, and deviations are extracted due to their direct influence on glitches. Based on the parameters, a random forest regression model is trained. [82] compares three different types of models to model glitch shape and power: linear regression, gradient boosting trees, and ANNs. Model

outputs are glitch height, glitch width, time-to-peak value, delay value, and total energy. [6] considers glitches by defining three different propagation types for timing and power based on input pulse width: no propagation if it is too small, no logic propagation but small power influence if it is wide enough to illicit power but not switching, and full propagation if the gate switches.

Evaluation: Different ground truths are used for evaluation: hardware measurements [10], transistor-level simulation [63], [82], gate-level power [55], [48], [6], post-P&R power [44]. [48] defines glitch power as the difference between zero-delay and delay gate-level simulations. [82] only uses ANN for final tests because linear regression performed worst overall and while GBT performs close to ANN the memory usage is higher.

The applications used for evaluation are:

- finite state machine with combination of combinational and sequential logic [10], [71]
- ISCAS85 benchmarks [63] (+ISCAS89), [55], [48] (+DesignWare), [6] (+AxBench)
- simple and complex gates [82]
- 5-ring oscillator [71]
- non-descript out-of-order CPU [44]

[44] compares to post-P&R power as ground truth and additionally uses commercial gate-level power estimates as a reference. Results with six different designs with varying gate sizes show improvements in accuracy (0.8% vs 25%) and simulation speed. This suggests that their model is able to circumvent the layout process without loss of accuracy.

The involvement of hardware parameters and timing effects on power, such as glitches, add another layer of complexity to power models. This can be seen in the choice of applications used for the verification, which are simpler and smaller than designs used in the purely switching-activity-based methods. Especially the methods that rely on custom characterization are limited. ML-based methods might simplify this process in the future as initial results from [82] show.

Lesson Learned from PMs at Gate-Level:

Use-Case: PMs at gate-level have the potential for higher overall accuracy and to capture effects that are mostly invisible at higher levels, such as glitches. Therefore, input features not only include switching activity but may also include hardware information obtained from a liberty file or custom characterization.

Types of Circuits: The differences in circuits types is highly influenced by the information density. On one hand, run-time models that only require switching activity as input test larger circuits up to SoCs and CPUs. On the other hand, models that included hardware information test only smaller circuits like FSMs or even just simple gates.

Types of Models: Run-time models use mostly linear regression, while models that include hardware information lean toward a more traditional approach, like analytical or LUTs, with just a few ML models.

Research Focus: Gate-level enables the usage of hardware information. What can be seen is that while liberty files are often instrumented, custom characterization is also done to model edge cases not present in liberty files. Additionally, timing is sometimes included in the model. Run-time models make use of reduction methods to lower the overhead of the hardware implementation.

Open-Source PMs: None of the considered works provided open-source code of their power model.

6 Discussion

Power modeling has remained an active and ongoing research area over the past decade. Figure 9 shows the reported maximal errors for the variety of modeling methods utilized at the different abstraction levels. The presented results, however, cannot be taken at face value. To identify the best model for a certain use-case, a fair comparison between methods is crucial. But multiple factors hinder easy comparison between methods and the evaluation of model accuracy. One factor is modeling different target applications that not only differ in terms of component size but also in terms of functionality (combinational vs sequential logic, threading, etc). This also includes which part of the power dissipation is modeled.

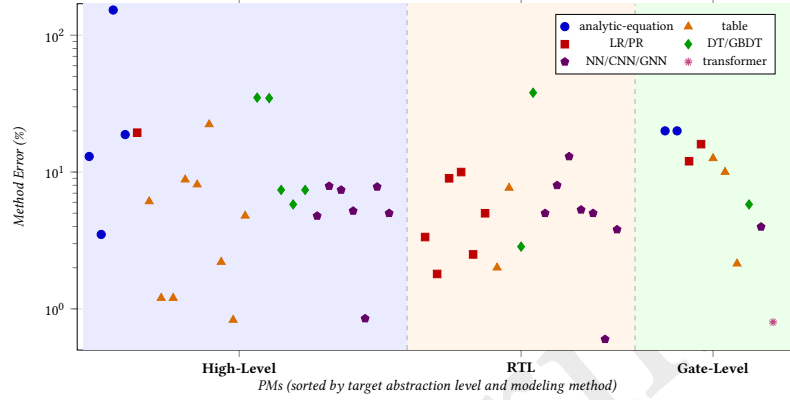


Fig. 9. Errors of PMs sorted by target abstraction levels and modeling method

Figure 10a shows the distribution of the targeted power sources. Here, *total* refers to works that have a single power model for total power while *dynamic + static* includes works that have a separate model for each source. The main research focus is on dynamic power. Even if total power is targeted, input features, such as SA, are mostly suited to variations in dynamic power. Another factor limiting comparability is that accuracy results (often expressed as error measurements) are highly influenced by the complexity of benchmarks and the novelty of technology nodes. While most methods used individual benchmark sets, with small overlap, a wide range of technology node sizes was considered. Different technologies show changes in power dissipation characteristics and corresponding challenges [92]. Figure 10b shows the spectrum of technology node sizes on which the PMs have been evaluated. Besides many methods that do not disclose this information, only a few methods have already considered recent technology nodes. Finally, there is no universal error measurement, and many different measurements are used across the works considered in this paper, as shown in Figure 10c. Additionally, the choice of ground truth varies greatly, as can be seen in Figure 11. While the majority of works instrument commercial tools for ground truth data generation, there still is a divide between the different providers, see Figure 10d. Comparison is also difficult because power model methods rarely build directly upon or compare results to previous works. Only 30% of the methods have been compared directly to a reference method.

Beyond these limiting factors, we provide a deep insight into the five key aspects of this SLR, as defined by the proposed RQs in Section 4, as follows:

- **Abstraction levels:** The different levels have different prerequisites and challenges, which lead to specific modeling approaches.
- **Machine learning:** ML-based power modeling methods get increasing attention and show promising results.

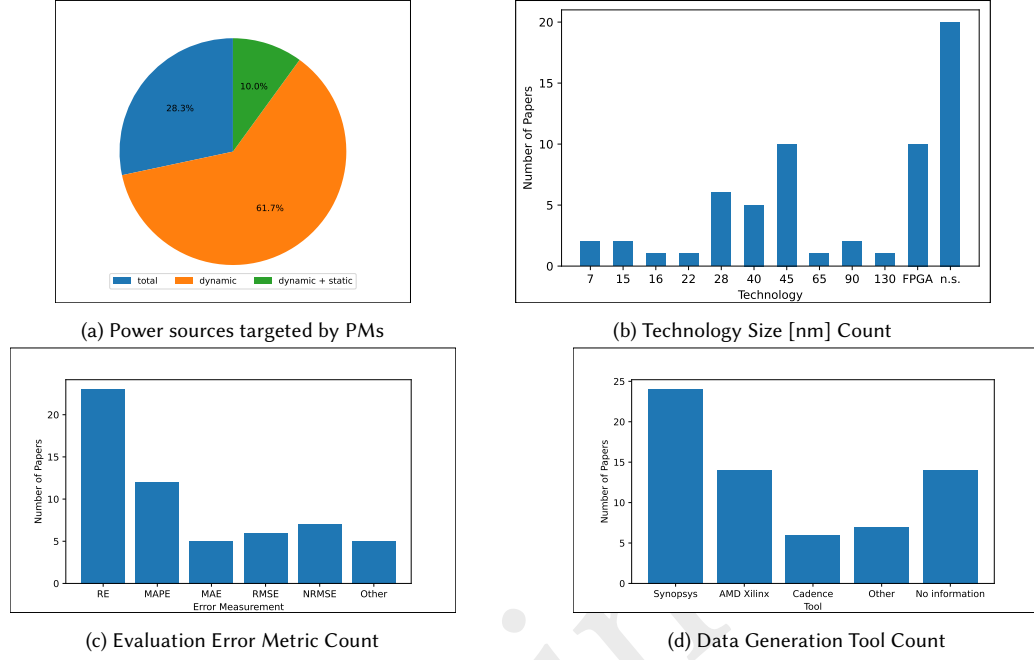


Fig. 10. Statistics on PM methods with respect to power source, technology sizes, evaluation error metrics and tools for data generation

- **Data origin and fidelity:** Especially for data-driven methods, like ML, the suitable choice of ground-truth power data is crucial, but requires a trade-off for the specific application.
- **Open-source:** Although beneficial for various aspects of PM research, only a minority of methods are provided open-source.
- **Open challenges:** PMs are investigated since decades. However, still various aspects, like model transferability or special circuit types, need further attention.

6.1 Comparison of Different Target Abstraction Levels

Table 6 presents an overview of the main characteristics of each abstraction level. The focus is on central aspects such as use-cases and input data, general research focus but also special aspects that stand out when compared to other levels and finally open challenges. Similarities can be seen in input data which is often switching activity and additional information available at each level. RTL is the most limited in terms of input variety. Use-cases are fairly similar in RTL and gate-level while high-level stands out with memory system power models. Additionally, high-level models that estimate power without HLS have been proposed. Only gate-level offers dedicated static power models. The research focus for all levels is to present a valuable alternative to commercial tools in terms of inference speed while still attaining comparable accuracy. A lot of RTL and gate-level works skew towards run-time power and include hardware implementations of their models. Run-time models are in general less complex and make use of data reduction techniques because of hardware constraints. Models at high-level are very application dependent as the input feature selection is strongly influenced by the use-case. Many methods directly interact with or make change to HLS-processes. A few models at RTL propose synthesis alternatives with graph-based models to circumvent the need for synthesis.

Table 6. Comparison of main aspects of the different abstraction levels

	High-Level	RTL	Gate-Level
Main Input	• Switching activity • Instructions • System Configuration • Control state	• Switching activity • Circuit in graph representation	• Switching activity • Liberty file • Timing
Use-Cases	• Design-time estimation • Dynamic power or total power models • Average and cycle-accurate PMs • CPU, SoC, memory systems, IP blocks • Hierarchical models for SoC • Estimation prior to HLS	• Design-time estimation • Run-time estimation • Dynamic or total power models • Average and time-based power models • CPU, SoC, small circuits	• Design-time estimation • Run-time estimation • Dynamic, static or total power • Average and time-based power models • CPU, SoC, small circuits
Research Focus	• Inference speed • High accuracy, although limited circuit information • CPU/ SoC power • Time-consuming data generation	• Inference speed • Run-time models with hardware implementation (overhead focused)	• Inference speed • Run-time models with hardware implementation (overhead focused) • Detailed modeling of side-effects (glitches)
Special Aspects	• Very application-specific input features • Simple (e.g., LUT-based) models • HLS-process-based models	• Transferability models explored • Reduction of input data for ML models • Graph-based methods as synthesis alternative	• Hardware information utilization • Timing consideration • Custom gate characterization • Reduction of input data for ML models
Open Topics	• Transferability • Standardized benchmarks • Special IC types (e.g., memories)	• Transferability • Standardized benchmarks • Method comparisons with same application	• Transferability • Standardized benchmarks • Method comparisons with same application • Custom characterization takes time

Gate-level models often require more detailed information than provided in the standard cell library and therefore undergo custom gate characterization. Additionally, timing effects are considered only at gate-level.

Across all levels the biggest challenges remain transferability of power models and standardized benchmarks for accuracy evaluation. Comparison data between methods is still lacking though a few comparison studies exist at RTL. High-level specifically faces difficulties due to special IC types such as memories which lack open-source information. A problem at gate-level is the custom characterization of gates which takes time and effort and has to be repeated for every technology.

6.2 Machine Learning

The most dominant common trend across high-Level, RTL, and gate-level is the usage of ML-based modeling methods (see Section 4 and Figure 4b). The methods have multiple advantages for power modeling, like the flexibility to be applied to different kinds of circuit input features, the ability to follow the inherent non-linear relationships to power and the potential for fast inference compared to commercial tools.

But ML methods often require a time-consuming training phase, especially NNs. Furthermore, a lot of training data is required. Data quality is crucial to the model's performance. Obtaining diverse data for a single circuit is already challenging to quantify (e.g., in terms of the coverage of the input workload space). Modeling average power requires a large number of simulation runs, or test benches have to be split into smaller sequences. Conversely, fine-grained data for cycle-accurate PM can easily reach large amounts, where not every clock cycle is useful for training. Here, especially at RTL, reducing data by removing redundant signals and sampling simulation cycles was investigated.

Commercial tools for power estimation are applicable to a variety of circuit designs. A major difficulty with ML models is their significant adaptation to a specific application and the transferability to other circuits or technologies. Besides the feature selection, which is targeted to specific circuits (like CPUs), obtaining large sets of realistic sample circuits is limited (especially in academia) due to the proprietary nature of circuit design. Approaches, like automatic (artificial) netlist generation [34], could support here, but need further research in terms of their effectiveness for improving the power model. Transferability of power models has been studied, but almost always includes retraining the model if not using a completely separate model. These approaches are, however, either still limited to specific classes of circuits or have only demonstrated their approach on a small set of benchmark designs. In particular, comparing a single, high-level CPU model with a composed model of the submodules shows a significant reduction in accuracy. Although transferability was not even tested, a PM that is designed to allow system changes already faces challenges in achieving high accuracy.

6.3 Data Origin and Fidelity

The chosen ground truth source especially influences the quality of training ML-based models (and other data-driven approaches, such as LUTs). Data obtained from lower design levels or from on-board measurements are closer to a circuit's actual behavior (e.g., due to detailed signal delays). However, design parameters such as technology node, operating conditions or special constraints are specified during the design process and directly influence the final circuit design. The fidelity of a model based on these data would be higher, but only for the specific IC setup. Further aspects, such as variations in hardware manufacturing or potential errors in on-board measurements, can limit the usefulness of theoretically precise data. Hence, providing a *golden* ground truth source for all PMs in training is not possible.

But the chosen ground-truth source may also affect the achievable accuracy in evaluating the PMs. Figure 11 illustrates the variety of levels of ground truth data that have been utilized. It can be seen that the methods across all target abstraction levels leverage information from all lower levels. High-level methods are often based on on-chip measurements, RTL-based PMs use gate-level information from various design stages (i.e., post-synthesis and post-P&R) and physical-level information is used by gate-level PMs. Furthermore, Figure 11 highlights the recorded accuracy (in the form of maximal error) for the PMs. No qualitative correlation between the abstraction levels and modeling error is observable. It is neither observable that a more precise data source leads to higher modeling accuracy (due to the higher precision of the data) nor to lower accuracy (due to more complexity). For a direct comparison of PMs, different ground truth sources must be accounted for in the accuracy evaluation (due to varying levels of data fidelity and specificity). As

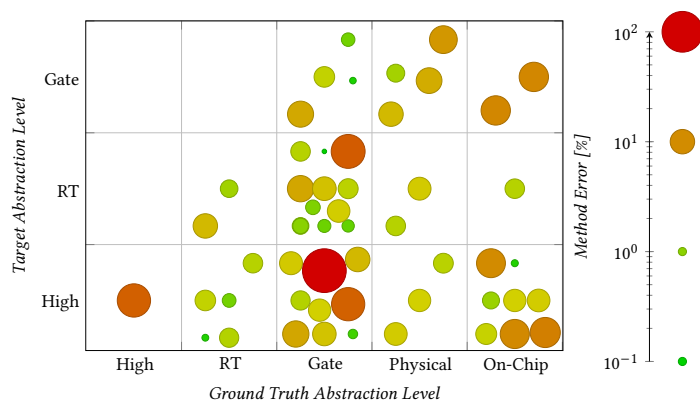


Fig. 11. Maximal errors of primary studies for target and ground truth abstraction levels

this is difficult to quantify with general validity, comparing PM methods across papers (even for similar application/abstraction levels) is complicated. Furthermore, accuracy is mainly measured with an error metric; however, a wide variety of different metrics is used, which hinders direct comparison. Accuracy is also influenced by using different granularities of the ground truth data (i.e., average power, multi-cycle, cycle-accurate, and even instantaneous) or by using different tools for power data generation. Even with the same ground-truth data and benchmarks, multiple synthesis and simulation parameters (e.g., HLS directives) can influence the final results and must be considered. Of the studies that disclosed the tools that were used, most methods relied on proprietary tools. Figure 10d shows an overview of the vendors, where the widespread use of Synopsys tools, such as PrimePower [1], is evident. Especially in comparative evaluations of methods, this dominance of commercial tools imposes another limitation.

6.4 Open-Source

Providing the PMs open-source provides multiple benefits, like easier evaluation of previous methods and easier progress to more refined methods. However, only 15% of the PM methods have published their model implementation as open-source. For the open-sourced ones, the code is often very limited and features only the training algorithm, which must be fed training data. A reason for the lack of open-source PMs can be the strong interrelation between modeling methods and proprietary tools (e.g., by features obtained directly from HLS tools at high-level). The dominance of proprietary circuit designs, technology information, and ground-truth power data may strongly limit the publication of training data. However, in other domains of IC design, open-source gets more attention in recent years (e.g., the RISC-V ISA [85] or the ASIC design flow [75]). Looking specifically at power modeling, open-source development within the hardware community remains very scarce compared to the software community (especially in research).

6.5 Open Challenges

Besides the open challenge of fully leveraging open-sourced power models, there exist multiple other open topics for future research:

- **Transferability:** Especially for ML-based PMs, transferability needs further attention. For the same design, not only is modeling different workloads important, but also modeling different technology size. A transferable PM would also be applicable to various designs. The generalizability of ML-based PMs to circuits outside the

training and test set has only been targeted in a limited way, e.g., by applying cross-evaluation schemes on a small number of benchmark circuits.

- *Benchmarks*: There are no specialized benchmarks for training and validation of power models and often performance benchmarks are used. Attempts at alternatives have been made with random benchmarks or genetic algorithms, but a proper study on the quality of benchmarks is missing. What makes benchmarks adequate to evaluate a power model?
- *Timing*: Precise timing has an evident influence on power dissipation estimates [54]. In the generation of ground truth data, most PM methods have not provided detailed information about how timing was considered (e.g., by a fully timing-annotated gate-level simulation or just functional, zero-delay information).
- *Power sources*: Even in the past decade, most PM methods have targeted either dynamic power directly or implicitly by their choice of input features (i.e., considering only SA features). However, with shrinking technology sizes, the impact of static power increases [17]. This is only barely visible in research across all design levels.
- *Large language models (LLMs)*: LLMs have recently received major attention in AI research in general, but also in EDA. While no current model focuses solely on power modeling, frameworks such as [18] include performance and area predictions at the RTL level. In a two-step process, various views of a circuit are encoded into an embedding in a self-supervised way, which is then applied to a small ML model trained with only a few samples for the target application. Depending on the encoder’s maturity, this could especially support the transferability of ML-based PMs and could therefore also be considered here.
- *GPU/ parallel processing/ memory*: Most PM methods focus on CPU or SoC designs with smaller IP blocks. But, in the era of AI, parallel processing units, like GPU, have got major attention. Similarly to memory devices, GPUs are highly proprietary which leads to only minor attempts of indirect power characterization and modeling (as in [51]).

7 Conclusion

This paper presented a systematic literature review (SLR) that offers a comprehensive overview of integrated-circuit power modeling research. We traced the field’s evolution over the past decade, focusing on two pillars: First, examining studies across the three principal abstraction levels, mainly high-level, RTL, and gate-level, and the other pillar is to cover a wide spectrum of power modeling approaches ranging from analytical to machine-learning-based. The results underscore that power estimation remains a critical concern in semiconductor design and continues to be highly relevant as the industry advances.

Our analysis yields five key insights. First, current research increasingly targets system-wide or component-level models, reflecting a trend toward higher abstraction. Here, CPUs dominate the surveyed works, indicating an enduring demand for accurate CPU power models. Second, machine-learning approaches are both widespread and often report the highest accuracy, signaling their growing centrality in IC power research. However, simpler methods often show similarly accurate results with less effort compared to ML. The modeling method proves to be a minor deciding factor in terms of accuracy compared to the selection of input features. Third, across all design stages, inference speed and accuracy are the biggest focus. Transferability of a model to a different technology or circuit is one of the most frequently observed limitations. However, the reported method accuracy shows no clear correlation with the chosen abstraction level or the ground-truth source. Fourth, unlike other IC design domains, power modeling relies heavily on ground-truth data from commercial tools and lacks open-source implementations. Fifth, although power modeling is well studied,

aspects such as memory modeling require further research. Here, we conclude our SLR by identifying three principal open challenges that must be addressed to accelerate progress. Accurate data are essential for machine learning, yet the choice of ground truth source is not always trivial and potentially biases learning. Models that perform well on one application frequently degrade when applied elsewhere. This lack of transferable models limits the practical applicability. Moreover, the lack of publicly available power models hampers reproducibility and slows innovation. Overcoming these obstacles will require coordinated efforts across academia and industry, coupled with rigorous methodological standards. By tackling these challenges, the community can harness the full potential of both traditional analytical methods and emerging data-driven techniques to deliver reliable, high-performance power models for next-generation semiconductor designs.

8 Acknowledgments

The work has been funded by the German Federal Ministry of Research, Technology and Space (BMFT) within the project DI-OSVISE (contract no. 16ME0957 | 16ME0958).

References

- [1] Synopsys, Inc. [n. d.]. *PrimePower Q-2019.12-SP1*. <https://www.synopsys.com/implementation-and-signoff/signoff/primepower.html>
- [2] Ahmed Abdel-Haleem and Magdy A. El-Moursy. 2017. TLM Virtual Platform for Fast and Accurate Power Estimation. In *2017 18th International Workshop on Microprocessor and SOC Test and Verification (MTV)*. 35–38. <https://doi.org/10.1109/MTV.2017.17>
- [3] Hamdi E. Ahmed, B.H. Salman, A.Sh. Kherbeet, and M.I. Ahmed. 2018. Optimization of thermal design of heat sinks: A review. *International Journal of Heat and Mass Transfer* 118 (2018), 129–153. <https://doi.org/10.1016/j.ijheatmasstransfer.2017.10.099>
- [4] Krste Asanovic, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, et al. 2016. The rocket chip generator. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17* 4 (2016), 6–2.
- [5] Amr Baher, Ahmed N. El-Zeiny, Ahmed Aly, Ahmed Khalil, Adham Hassan, AbdelRahman Saeed, Karim Abo El Makarem, Magdy El-Moursy, and Hassan Mostafa. 2018. Dynamic power estimation using Transaction Level Modeling. *Microelectronics Journal* 81 (2018), 107–116. <https://doi.org/10.1016/j.mejo.2018.08.012>
- [6] Katayoon Basharkhah and Zainalabedin Navabi. 2024. Event-Based Power Analysis Integrated with Timing Characterization and Logic Simulation. In *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, Knoxville, TN, USA, 684–688. <https://doi.org/10.1109/ISVLSI61997.2024.00131>
- [7] Cem Benar, George Phan, Sylvia Chan, Zongfang Lin, Yat Fai Lam, and Robert Chu. 2024. PACE: MLP-Based Fast and Accurate Per-Cycle Chip Power Modelling. In *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, Knoxville, TN, USA, 689–693. <https://doi.org/10.1109/ISVLSI61997.2024.00132>
- [8] Christopher Celio, David A Patterson, and Krste Asanovic. 2015. The berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized risc-v processor. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167* (2015).
- [9] Saumya Chandra, Ramkumar Jayaseelan, and Ravi Bhargava. 2016. Speculative path power estimation using trace-driven simulations during high-level design phase. In *2016 IEEE 34th International Conference on Computer Design (ICCD)*. 630–637. <https://doi.org/10.1109/ICCD.2016.7753350>
- [10] Verginia-Iulia-Maria Chereja, Adriana-Ioana Potarniche, Sergiu-Alex Ranga, Botond Sandor Kirei, and Marina Dana Topa. 2018. Power Dissipation Estimation of CMOS Digital Circuits at the Gate Level in VHDL. In *2018 International Symposium on Electronics and Telecommunications (ISETC)*. IEEE, Timisoara, Romania, 1–4. <https://doi.org/10.1109/ISETC.2018.8583957>
- [11] Amit Chhabra, Harsh Rawat, Mohit Jain, Pascal Tessier, Daniel Pierredon, Laurent Bergher, and Promod Kumar. 2015. FALPEM: Framework for Architectural-Level Power Estimation and Optimization for Large Memory Sub-Systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 34, 7 (2015), 1138–1142. <https://doi.org/10.1109/TCAD.2014.2387859>
- [12] Maxime Colmant, Mascha Kurpicz, Pascal Felber, Loïc Huertas, Romain Rouvoy, and Anita Sobe. 2015. Process-level power estimation in VM-based systems. In *Proceedings of the Tenth European Conference on Computer Systems (Bordeaux, France) (EuroSys '15)*. Association for Computing Machinery, New York, NY, USA, Article 14, 14 pages. <https://doi.org/10.1145/2741948.2741971>
- [13] Luca Cremona, William Fornaciari, and Davide Zoni. 2021. Automatic identification and hardware implementation of a resource-constrained power model for embedded systems. *Sustainable Computing: Informatics and Systems* 29 (2021), 100467.
- [14] Amr B. Darwish, Magdy A. El-Moursy, and Mohamed Dessouky. 2016. Transaction Level Power Modeling (TLPM) Methodology. In *2016 17th International Workshop on Microprocessor and SOC Test and Verification (MTV)*. 61–64. <https://doi.org/10.1109/MTV.2016.21>
- [15] James J. Davis, Eddie Hung, Joshua M. Levine, Edward A. Stott, Peter Y. K. Cheung, and George A. Constantinides. 2018. KAPow: High-Accuracy, Low-Overhead Online Per-Module Power Estimation for FPGA Designs. *ACM Trans. Reconfigurable Technol. Syst.* 11, 1, Article 2 (Jan. 2018), 22 pages.

- <https://doi.org/10.1145/3129789>
- [16] Chenchen Deng, Leibo Liu, Yang Liu, Shouyi Yin, and Shaojun Wei. 2017. PMCC: Fast and Accurate System-Level Power Modeling for Processors on Heterogeneous SoC. *IEEE Transactions on Circuits and Systems II: Express Briefs* 64, 5 (2017), 540–544. <https://doi.org/10.1109/TCSII.2016.2615930>
 - [17] Ndubuisi Ekeke and Ralph Etienne-Cummings. 2006. Power dissipation sources and possible control techniques in ultra deep submicron CMOS technologies. *Microelectronics Journal* 37, 9 (2006), 851–860. <https://doi.org/10.1016/j.mejo.2006.03.008>
 - [18] Wenji Fang, Shang Liu, Hongce Zhang, and Zhiyao Xie. 2025. A Self-Supervised, Pre-Trained, and Cross-Stage-Aligned Circuit Encoder Provides a Foundation for Various Design Tasks. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference (Tokyo, Japan) (ASPAC '25)*. Association for Computing Machinery, New York, NY, USA, 505–512. <https://doi.org/10.1145/3658617.3697597>
 - [19] Wenji Fang, Yao Lu, Shang Liu, Qijun Zhang, Ceyu Xu, Lisa Wu Wills, Hongce Zhang, and Zhiyao Xie. 2023. MasterRTL: A Pre-Synthesis PPA Estimation Framework for Any RTL Design. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, San Francisco, CA, USA, 1–9. <https://doi.org/10.1109/ICCAD57390.2023.10323951>
 - [20] Wenji Fang, Yao Lu, Shang Liu, Qijun Zhang, Ceyu Xu, Lisa Wu Wills, Hongce Zhang, and Zhiyao Xie. 2025. Transferable Presynthesis PPA Estimation for RTL Designs With Data Augmentation Techniques. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 1 (Jan. 2025), 200–213. <https://doi.org/10.1109/TCAD.2024.3420904>
 - [21] Luis Francisco, Paul Franzone, and W. Rhett Davis. 2021. Fast and Accurate PPA Modeling with Transfer Learning. In *2021 ACM/IEEE 3rd Workshop on Machine Learning for CAD (MLCAD)*. 1–6. <https://doi.org/10.1109/MLCAD52597.2021.9531109>
 - [22] Andrea Galimberti, Michele Piccoli, and Davide Zoni. 2024. Blink: Fast Automated Design of Run-Time Power Monitors on FPGA-Based Computing Platforms. In *2024 31st IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. 1–4. <https://doi.org/10.1109/ICECS61496.2024.10849220>
 - [23] Ian Grout. 2008. *Digital Systems Design with FPGAs and CPLDs*. Newnes, USA.
 - [24] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7e9-Paper.pdf
 - [25] Yuko Hara, Hiroyuki Tomiyama, Shinya Honda, Hiroaki Takada, and Katsuya Ishii. 2008. CHStone: A benchmark program suite for practical C-based high-level synthesis. In *2008 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1192–1195. <https://doi.org/10.1109/ISCAS.2008.4541637>
 - [26] Ahmed Hemani, Nasim Farahini, Syed M. A. H. Jafri, Hassan Sohofi, Shuo Li, and Kolin Paul. 2017. *The SiLago Solution: Architecture and Design Methods for a Heterogeneous Dark Silicon Aware Coarse Grain Reconfigurable Fabric*. Springer International Publishing, Cham, 47–94. https://doi.org/10.1007/978-3-319-31596-6_3
 - [27] S. Hesselbarth, G. Schewior, and H. Blume. 2017. FPGA emulation methodology for fast and accurate power estimation of embedded processors. *Journal of Systems Architecture* 77 (2017), 14–25. <https://doi.org/10.1016/j.sysarc.2016.12.008>
 - [28] Zih-Ci Huang, Chi-Kang Chen, and Ren-Song Tsay. 2015. AROMA: A highly accurate microcomponent-based approach for embedded processor power analysis. In *The 20th Asia and South Pacific Design Automation Conference*. 761–766. <https://doi.org/10.1109/ASPAC.2015.7059102>
 - [29] Syed Mohammad Asad Hassan Jafri, Nasim Farahini, and Ahmed Hemani. 2017. SiLago-CoG: Coarse-Grained Grid-Based Design for Near Tape-Out Power Estimation Accuracy at High Level. In *2017 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. 25–31. <https://doi.org/10.1109/ISVLSI.2017.15>
 - [30] Jianlei Yang, Liwei Ma, Kang Zhao, Yici Cai, and Tin-Fook Ngai. 2015. Early Stage Real-Time SoC Power Estimation Using RTL Instrumentation. In *The 20th Asia and South Pacific Design Automation Conference*. IEEE, Chiba, Japan, 779–784. <https://doi.org/10.1109/ASPAC.2015.7059105>
 - [31] Staffs Keele et al. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report. Technical report, ver. 2.3 ebse technical report. ebse.
 - [32] Ben Keller. 2013. Risc-v, spike, and the rocket core. *Berkeley Architecture Group* (2013).
 - [33] Donggyu Kim, Adam Izraelevitz, Christopher Celio, Hokeun Kim, Brian Zimmer, Yunsup Lee, Jonathan Bachrach, and Krste Asanovic. 2016. Strober: Fast and Accurate Sample-Based Energy Simulation for Arbitrary RTL. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, Seoul, South Korea, 128–139. <https://doi.org/10.1109/ISCA.2016.21>
 - [34] Daeyeon Kim, Hyunjeong Kwon, Sung-Yun Lee, Seungwon Kim, Mingyu Woo, and Seokhyeong Kang. 2021. Machine Learning Framework for Early Routability Prediction with Artificial Netlist Generator. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1809–1814. <https://doi.org/10.23919/DAT51398.2021.9473966>
 - [35] Donggyu Kim, Jerry Zhao, Jonathan Bachrach, and Krste Asanovic. 2019. Simmani: Runtime Power Modeling for Arbitrary RTL with Automatic Signal Selection. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. ACM, Columbus OH USA, 1050–1062. <https://doi.org/10.1145/3352460.3358322>
 - [36] Botond Sandor Kirei, V.-I.-M Chereja, S. Hintea, and Marina Topa. 2019. PAELib: A VHDL Library for Area and Power Dissipation Estimation of CMOS Logic Circuits. *Advances in Electrical and Computer Engineering* 19 (01 2019), 9–16. <https://doi.org/10.4316/AECE.2019.01002>
 - [37] Johannes Knödtel, Wolffhardt Schwabe, Tobias Lieske, Marc Reichenbach, and Dietmar Fey. 2018. A Novel Methodology for Evaluating the Energy Consumption of IP Blocks in System-Level Designs. In *2018 28th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS)*. 46–53. <https://doi.org/10.1109/PATMOS.2018.8464149>
 - [38] Ajay Krishna Ananda Kumar and Andreas Gerstlauer. 2019. Learning-Based CPU Power Modeling. In *2019 ACM/IEEE 1st Workshop on Machine Learning for CAD (MLCAD)*. 1–6. <https://doi.org/10.1109/MLCAD48534.2019.9142100>

- [39] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004*. 75–86. <https://doi.org/10.1109/CGO.2004.1281665>
- [40] Dongwook Lee and Andreas Gerstlauer. 2018. Learning-Based, Fine-Grain Power Modeling of System-Level Hardware IPs. *ACM Trans. Des. Autom. Electron. Syst.* 23, 3, Article 30 (Feb. 2018), 25 pages. <https://doi.org/10.1145/3177865>
- [41] Wooseok Lee, Youngchun Kim, Jee Ho Ryoo, Dam Sunwoo, Andreas Gerstlauer, and Lizy K. John. 2015. PowerTrain: A learning-based calibration of McPAT power models. In *2015 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. 189–194. <https://doi.org/10.1109/ISLPED.2015.7273512>
- [42] Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, and Norman P. Jouppi. 2009. McPAT: an integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture (New York, New York) (MICRO 42)*. Association for Computing Machinery, New York, NY, USA, 469–480. <https://doi.org/10.1145/1669112.1669172>
- [43] Shuo Li, Nasim Farahini, Ahmed Hemani, Kathrin Rosvall, and Ingo Sander. 2013. System level synthesis of hardware for DSP applications using pre-characterized function implementations. In *2013 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*. 1–10. <https://doi.org/10.1109/CODES-ISSS.2013.6659003>
- [44] Wenkai Li, Yao Lu, Wenji Fang, Jing Wang, Qijun Zhang, and Zhiyao Xie. 2025. ATLAS: A Self-Supervised and Cross-Stage Netlist Power Model for Fine-Grained Time-Based Layout Power Analysis. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, San Francisco, CA, USA, 1–7. <https://doi.org/10.1109/DAC63849.2025.11132751>
- [45] Zhe Lin, Zike Yuan, Jieru Zhao, Wei Zhang, Hui Wang, and Yonghong Tian. 2022. PowerGear: Early-Stage Power Estimation in FPGA HLS via Heterogeneous Edge-Centric GNNs. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1341–1346. <https://doi.org/10.23919/DATE54114.2022.9774682>
- [46] Zhe Lin, Wei Zhang, and Sharad Sinha. 2017. Decision Tree Based Hardware Power Monitoring for Run Time Dynamic Power Management in FPGA. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. 1–8. <https://doi.org/10.23919/FPL.2017.8056832>
- [47] Zhe Lin, Jieru Zhao, Sharad Sinha, and Wei Zhang. 2020. HL-Pow: A Learning-Based Power Modeling Framework for High-Level Synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 574–580. <https://doi.org/10.1109/ASP-DAC47756.2020.9045442>
- [48] Zilu Liu, Ningmei Yu, and Haojie Si. 2022. Research on Glitch Power Estimation of Combinational Logic Circuits. In *2022 7th International Conference on Integrated Circuits and Microsystems (ICICM)*. IEEE, Xi'an, China, 37–40. <https://doi.org/10.1109/ICICM56102.2022.10011376>
- [49] Yao Lu, Qijun Zhang, and Zhiyao Xie. 2024. Unleashing Flexibility of ML-based Power Estimators Through Efficient Development Strategies. In *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design*. ACM, Newport Beach CA USA, 1–6. <https://doi.org/10.1145/3665314.3670823>
- [50] Yi-Chen Lu, Wei-Ting Chan, Vishal Khandelwal, and Sung Kyu Lim. 2022. Driving Early Physical Synthesis Exploration through End-of-Flow Total Power Prediction. In *Proceedings of the 2022 ACM/IEEE Workshop on Machine Learning for CAD*. ACM, Virtual Event China, 97–102. <https://doi.org/10.1145/3551901.3556476>
- [51] Jan Lucas and Ben Juurlink. 2016. ALUPower: Data Dependent Power Consumption in GPUs. In *2016 IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*. 95–104. <https://doi.org/10.1109/MASCOTS.2016.21>
- [52] Ahmed Mahmoudi, Andrija Nešković, Celine Thermann, Robin Sehm, Christoph Hübner, Tavia Plattenteich, Rolf Meyer, Rainer Buchty, Mladen Berekovic, and Saleh Mulhem. 2025. A Systematic Mapping Study on SystemC/TLM Modeling Capabilities in New Research Domains. *ACM Transactions on Design Automation of Electronic Systems* 30, 4 (2025), 1–41.
- [53] Mariem Makni, Smail Niar, Mouna Baklouti, and Mohamed Abid. 2018. HAPE: A high-level area-power estimation framework for FPGA-based accelerators. *Microprocessors and Microsystems* 63 (2018), 11–27. <https://doi.org/10.1016/j.micpro.2018.08.004>
- [54] Michael Meixner and Tobias G. Noll. 2014. Limits of gate-level power estimation considering real delay effects and glitches. In *2014 International Symposium on System-on-Chip (SoC)*. 1–7. <https://doi.org/10.1109/ISSOC.2014.6972437>
- [55] Michael Meixner and Tobias G. Noll. 2017. Accurate Estimation of CMOS Power Consumption Considering Glitches by Using Waveform Lookup. *IEEE Transactions on Circuits and Systems II: Express Briefs* 64, 7 (July 2017), 787–791. <https://doi.org/10.1109/TCSII.2016.2642179>
- [56] Mohamad Najem, Pascal Benoit, Mohamad El Ahmad, Gilles Sassatelli, and Lionel Torres. 2017. A Design-Time Method for Building Cost-Effective Run-Time Power Monitoring. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 36, 7 (July 2017), 1153–1166. <https://doi.org/10.1109/TCAD.2016.2614347>
- [57] Yehya Nasser, Jordane Lorandel, Jean-Christophe Prevotet, and Maryline Helard. 2021. RTL to Transistor Level Power Modeling and Estimation Techniques for FPGA and ASIC: A Survey. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 3 (March 2021), 479–493. <https://doi.org/10.1109/TCAD.2020.3003276>
- [58] Yehya Nasser, Jean-Christophe Prevotet, and Maryline Hélar. 2018. Power Modeling on FPGA: A Neural Model for RT-level Power Estimation. In *Proceedings of the 15th ACM International Conference on Computing Frontiers*. ACM, Ischia Italy, 309–313. <https://doi.org/10.1145/3203217.3204462>
- [59] Yehya Nasser, Carlo Sau, Jean-Christophe Prevotet, Tiziana Fanni, Francesca Palumbo, Maryline Hélar, and Luigi Raffo. 2019. NeuPow: artificial neural networks for power and behavioral modeling of arithmetic components in 45nm ASICs technology. In *Proceedings of the 16th ACM International Conference on Computing Frontiers (Alghero, Italy) (CF '19)*. Association for Computing Machinery, New York, NY, USA, 183–189. <https://doi.org/10.1145/3310273.3322820>

- [60] Yehya Nasser, Carlo Sau, Jean-Christophe Prévotet, Tiziana Fanni, Francesca Palumbo, Maryline H  lard, and Luigi Raffo. 2020. NeuPow: A CAD Methodology for High-level Power Estimation Based on Machine Learning. *ACM Transactions on Design Automation of Electronic Systems* 25, 5 (Sept. 2020), 29. <https://doi.org/10.1145/3388141>
- [61] Andrija Ne  kovi  , Saleh Mulhem, Alexander Treff, Rainer Buchty, Thomas Eisenbarth, and Mladen Berekovic. 2023. SystemC model of power side-channel attacks against AI accelerators: Superstition or not?. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–8.
- [62] Andrija Ne  kovi  , Ahmad Kabour, Mohamed Eltantawi, Mladen Berekovic, Rolf Meyer, and Saleh Mulhem. 2025. SCA-WAL: An Open-Source Framework for Power Side Channel Assessment at RTL and Netlist Level. *IEEE Embedded Systems Letters* (2025), 1–1. <https://doi.org/10.1109/LES.2025.3644491>
- [63] Alejandro Nocua, Arnaud Virazel, Alberto Bosio, Patrick Girard, and Cyril Chevalier. 2016. A Hybrid Power Estimation Technique to Improve IP Power Models Quality. In *2016 IFIP/IEEE International Conference on Very Large Scale Integration (VLSI-SoC)*. IEEE, Tallinn, Estonia, 1–6. <https://doi.org/10.1109/VLSI-SoC.2016.7753582>
- [64] Daniele Jahier Pagliari, Valentino Peluso, Yukai Chen, Andrea Calimera, Enrico Macii, and Massimo Poncino. 2018. All-Digital Embedded Meters for on-Line Power Estimation. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Dresden, Germany, 737–742. <https://doi.org/10.23919/DATE.2018.8342105>
- [65] Louis-No  l Pouchet et al. 2012. Polybench: The polyhedral benchmark suite. URL: <http://www.cs.ucla.edu/pouchet/software/polybench> 437 (2012), 1–1.
- [66] Jan M Rabaey, Anantha Chandrakasan, and Borivoje Nikolic. 2002. *Digital integrated circuits*. Vol. 2. Prentice hall Englewood Cliffs.
- [67] Martin Rapp, Hussam Amrouch, Yibo Lin, Bei Yu, David Z. Pan, Marilyn Wolf, and J  rg Henkel. 2022. MLCAD: A Survey of Research in Machine Learning for CAD Keynote Paper. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 10 (2022), 3162–3181. <https://doi.org/10.1109/TCAD.2021.3124762>
- [68] Joseph Raskind, Timur Babakol, Khaled Mahmoud, and Yu David Liu. 2024. VESTA: Power Modeling with Language Runtime Events. *Proc. ACM Program. Lang.* 8, PLDI, Article 172 (June 2024), 26 pages. <https://doi.org/10.1145/3656402>
- [69] Brandon Reagen, Robert Adolf, Yakun Sophia Shao, Gu-Yeon Wei, and David Brooks. 2014. MachSuite: Benchmarks for accelerator design and customized architectures. In *2014 IEEE International Symposium on Workload Characterization (IISWC)*. 110–119. <https://doi.org/10.1109/IISWC.2014.6983050>
- [70] Majdi Richa, Jean-Christophe Pr  votet, Micka  l Dardaillon, Mohamad Mrou  , and Abed Ellatif Samhat. 2022. High-Level Early Power Estimation of FPGA IP Based on Machine Learning. In *2022 29th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. 1–4. <https://doi.org/10.1109/ICECS202256217.2022.9970952>
- [71] Kirei Botond Sandor, Calin Adrian Farcas, and Marina Dana Topa. 2020. PAElib 2.0: Power&Area Aware Modeling of CMOS Digital Circuits in VHDL. In *2020 43rd International Conference on Telecommunications and Signal Processing (TSP)*. IEEE, Milan, Italy, 333–336. <https://doi.org/10.1109/TSP49548.2020.9163455>
- [72] Philipp Schlicker and Oliver Bringmann. 2020. Gate-Level Models for Fast Cross-Level Power Density Estimation. In *Proceedings of the 2020 on Great Lakes Symposium on VLSI*. ACM, Virtual Event China, 507–512. <https://doi.org/10.1145/3386263.3406911>
- [73] Stefan Sch  rmans, Gereon Onnebrink, Rainer Leupers, Gerd Ascheid, and Xiaotao Chen. 2016. Frequency-Aware ESL Power Estimation for ARM Cortex-A9 Using a Black Box Processor Model. *ACM Trans. Embed. Comput. Syst.* 16, 1, Article 26 (Oct. 2016), 26 pages. <https://doi.org/10.1145/2987375>
- [74] Prianka Sengupta, Aakash Tyagi, Yiran Chen, and Jiang Hu. 2022. How Good Is Your Verilog RTL Code?: A Quick Answer from Machine Learning. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. ACM, San Diego California, 1–9. <https://doi.org/10.1145/3508352.3549375>
- [75] Mohamed Shalan and Tim Edwards. 2020. Building OpenLANE: A 130nm OpenROAD-based Tapeout- Proven Flow : Invited Paper. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–6.
- [76] Jonathas Silveira, Lucas Castro, Victor Ara  jo, Rodrigo Zeli, Daniel Lazari, Marcelo Guedes, Rodolfo Azevedo, and Lucas Wanner. 2022. Prof5: A RISC-V profiler tool. In *2022 IEEE 34th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. 201–210. <https://doi.org/10.1109/SBAC-PAD55451.2022.00031>
- [77] Efsthios Sotiriou-Xanthopoulos, G. Shalina Percy Delicia, Peter Figuli, Kostas Siozios, George Economakos, and J  rgen Becker. 2015. A power estimation technique for cycle-accurate higher-abstraction SystemC-based CPU models. In *2015 International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*. 70–77. <https://doi.org/10.1109/SAMOS.2015.7363661>
- [78] Lukas Steiner, Thomas Psota, Marco M  rz, Derek Christ, Matthias Jung, and Norbert Wehn. 2025. DRAMPower 5: An Open-Source Power Simulator for Current Generation DRAM Standards. In *Proceedings of the Rapid Simulation and Performance Evaluation for Design Workshop (RAPIDO '25)*. Association for Computing Machinery, New York, NY, USA, 8–16. <https://doi.org/10.1145/3721848.3721850>
- [79] Xiaoyu Sun, Xiaochen Peng, Sai Qian Zhang, Jorge Gomez, Win-San Khwa, Syed Shakib Sarwar, Ziyun Li, Weidong Cao, Zhao Wang, Chiao Liu, Meng-Fan Chang, Barbara De Salvo, Kerem Akarvardar, and H.-S. Philip Wong. 2024. Estimating Power, Performance, and Area for On-Sensor Deployment of AR/VR Workloads Using an Analytical Framework. *ACM Trans. Des. Autom. Electron. Syst.* 29, 6, Article 93 (Sept. 2024), 27 pages. <https://doi.org/10.1145/3670404>
- [80] Abhishek N. Tripathi and Arvind Rajawat. 2018. Fast and efficient power estimation model for FPGA based designs. *Microprocessors and Microsystems* 59 (2018), 37–45. <https://doi.org/10.1016/j.micpro.2018.03.005>

- [81] Abhishek N. Tripathi and Arvind Rajawat. 2020. An Accurate and Quick ANN-Based System-Level Dynamic Power Estimation Model Using LLVM IR Profiling for FPGA Designs. *IEEE Embedded Systems Letters* 12, 2 (2020), 58–61. <https://doi.org/10.1109/LES.2019.2935052>
- [82] Anastasis Vagenas, Dimitrios Garyfallou, Nestor Evmorfopoulos, and George Stamoulis. 2024. Advanced Gate-Level Glitch Modeling Using ANNs. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. ACM, San Francisco CA USA, 1–6. <https://doi.org/10.1145/3649329.3657351>
- [83] Matthew J. Walker, Stephan Diestelhorst, Andreas Hansson, Anup K. Das, Sheng Yang, Bashir M. Al-Hashimi, and Geoff V. Merrett. 2017. Accurate and Stable Run-Time Power Modeling for Mobile and Embedded CPUs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 36, 1 (2017), 106–119. <https://doi.org/10.1109/TCAD.2016.2562920>
- [84] Zhiping Wang and W. Rhett Davis. 2021. An Instruction-Level Power and Energy Model for the Rocket Chip Generator. In *2021 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. 1–6. <https://doi.org/10.1109/ISLPED52811.2021.9502485>
- [85] Andrew Waterman, Krste Asanović, and John Hauser. 2026. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture* (document version 20260120 ed.). RISC-V International. <https://riscv.org/technical/specifications/> Official Release.
- [86] Zhiyao Xie, Shiyu Li, Mingyuan Ma, Chen-Chia Chang, Jingyu Pan, Yiran Chen, and Jiang Hu. 2022. DEEP: Developing Extremely Efficient Runtime On-Chip Power Meters. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. ACM, San Diego California, 1–9. <https://doi.org/10.1145/3508352.3549427>
- [87] Zhiyao Xie, Xiaoqing Xu, Matt Walker, Joshua Knebel, Kumaraguru Palaniswamy, Nicolas Hebert, Jiang Hu, Huanrui Yang, Yiran Chen, and Shidhartha Das. 2021. APOLLO: An Automated Power Modeling Framework for Runtime Power Introspection in High-Volume Commercial Microprocessors. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. ACM, Virtual Event Greece, 14. <https://doi.org/10.1145/3466752.3480064>
- [88] Ceyu Xu, Chris Kjellqvist, and Lisa Wu Wills. 2022. SNS’s Not a Synthesizer: A Deep-Learning-Based Synthesis Predictor. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. ACM, New York New York, 847–859. <https://doi.org/10.1145/3470496.3527444>
- [89] Jianwang Zhai, Yici Cai, and Bei Yu. 2023. Microarchitecture Power Modeling via Artificial Neural Network and Transfer Learning. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference* (Tokyo, Japan) (ASPAC ’23). Association for Computing Machinery, New York, NY, USA, 302–307. <https://doi.org/10.1145/3566097.3567844>
- [90] Qijun Zhang, Mengming Li, Yao Lu, and Zhiyao Xie. 2025. FirePower: Towards a Foundation with Generalizable Knowledge for Architecture-Level Power Modeling. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference* (Tokyo, Japan) (ASPAC ’25). Association for Computing Machinery, New York, NY, USA, 1145–1152. <https://doi.org/10.1145/3658617.3697554>
- [91] Qijun Zhang, Shiyu Li, Guanglei Zhou, Jingyu Pan, Chen-Chia Chang, Yiran Chen, and Zhiyao Xie. 2023. PANDA: Architecture-Level Power Evaluation by Unifying Analytical and Machine Learning Solutions. <https://doi.org/10.48550/arXiv.2312.08994>
- [92] Qingzhu Zhang, Yongkui Zhang, Yanna Luo, and Huaxiang Yin. 2024. New structure transistors for advanced technology node CMOS ICs. *National Science Review* 11, 3 (01 2024), nwae008. <https://doi.org/10.1093/nsr/nwae008> arXiv:<https://academic.oup.com/nsr/article-pdf/11/3/nwae008/56748859/nwae008.pdf>
- [93] Yuan Zhou, Haoxing Ren, Yanqing Zhang, Ben Keller, Brucek Khailany, and Zhiru Zhang. 2019. PRIMAL: Power Inference using Machine Learning. In *Proceedings of the 56th Annual Design Automation Conference 2019 (DAC ’19)*. Association for Computing Machinery, New York, NY, USA, Article 39, 6 pages. <https://doi.org/10.1145/3316781.3317884>
- [94] Davide Zoni, Luca Cremona, Alessandro Cilaro, Mirko Gagliardi, and William Fornaciari. 2018. PowerTap: All-digital Power Meter Modeling for Run-Time Power Monitoring. *Microprocessors and Microsystems* 63 (Nov. 2018), 12. <https://doi.org/10.1016/j.micpro.2018.07.007>
- [95] Davide Zoni, Luca Cremona, and William Fornaciari. 2018. PowerProbe: Run-time Power Modeling through Automatic RTL Instrumentation. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Dresden, 6. <https://doi.org/10.23919/DATE.2018.8342106>
- [96] Davide Zoni, Andrea Galimberti, and William Fornaciari. 2023. A Survey on Run-time Power Monitors at the Edge. *Comput. Surveys* 55, 14s (Dec. 2023), 1–33. <https://doi.org/10.1145/3593044>